Standard Architecture Views Primer

Contents

Standard Architecture Views 1

System Context 3

Component Architecture 3

Step 1: Component Architecture (Solution Sketch) 4

Step 2: Component Architecture (Full View) 6

Data Flow Diagram 7

Conceptual Data Model 8

Infrastructure Architecture 8

Deployment Architecture 9

Cloud Architecture 10

Security Architecture 13

Security Domains, Categories, Services 13

Data Centric Security 16

Secure by Design with Threat Modeling 18

Zero Trust Architecture 18

Application Security – Use Case (Example) 22

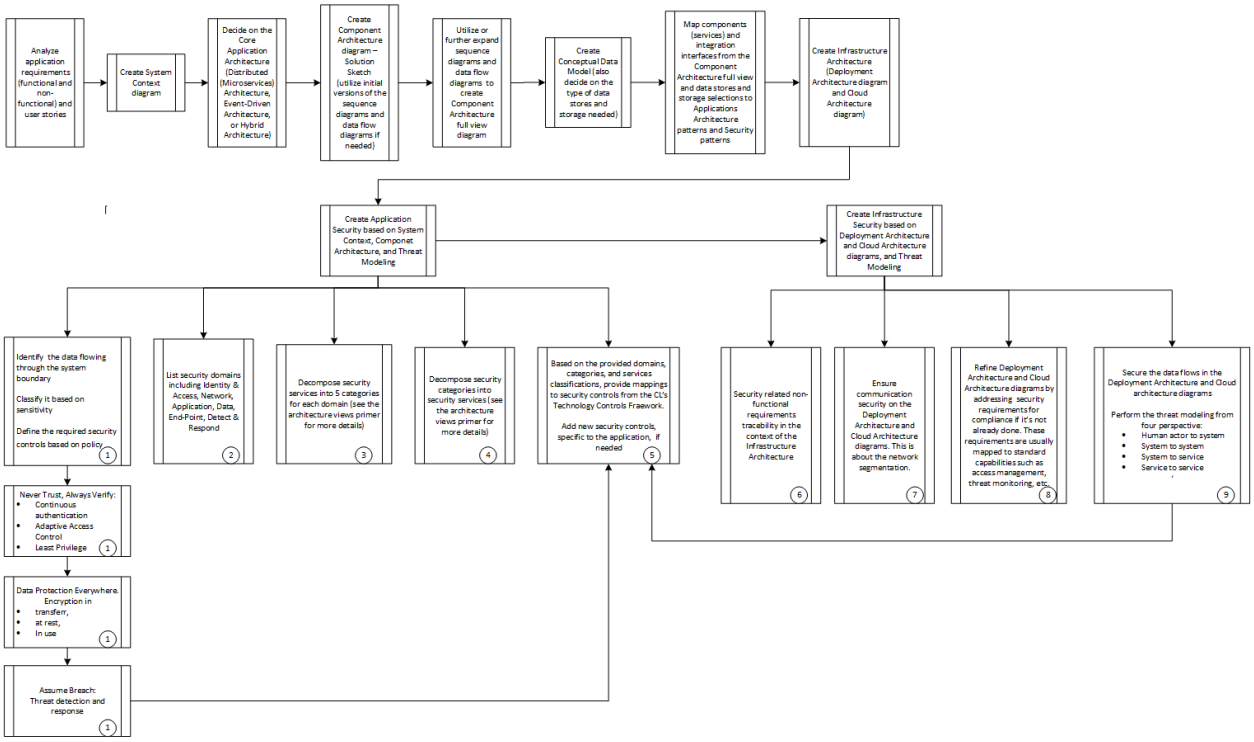
Infrastructure Security 36

Standard Architecture Views

The list of standard architecture views include:

- System Context
- Component Architecture (Solution Sketch)
- Sequence Diagram(s)
- Component Architecture (Full View)
- Data Flow Diagram(s)
- Conceptual Data Model
- Infrastructure Architecture
    - Deployment Architecture View
    - Cloud Archiutecture View
- Security Architecture
    - Security Domains, Categories, Services

- Zero Trust Architecture
  - Data Centric Security
    - System Context
    - Application Security
    - Infrastructure Security
  - Never Trust, Always Verify
    - Continuous authentication
    - Adaptive access control
    - Least privilege
    - Micro-segmentation
  - Data Protection Everywhere
    - Encryption in transit, at rest, in use
  - Assume Breach
    - Secure by Design with Threat Modeling (threat detection and response)
    - Mapping of security controls to security architecture
- Architecture Patterns
  - Map application architecture, components (services), and interfaces to architecture patterns
  - Map security services to security patterns
  - Map architecture patterns (including security patterns) to cloud and enterprise services



System Context

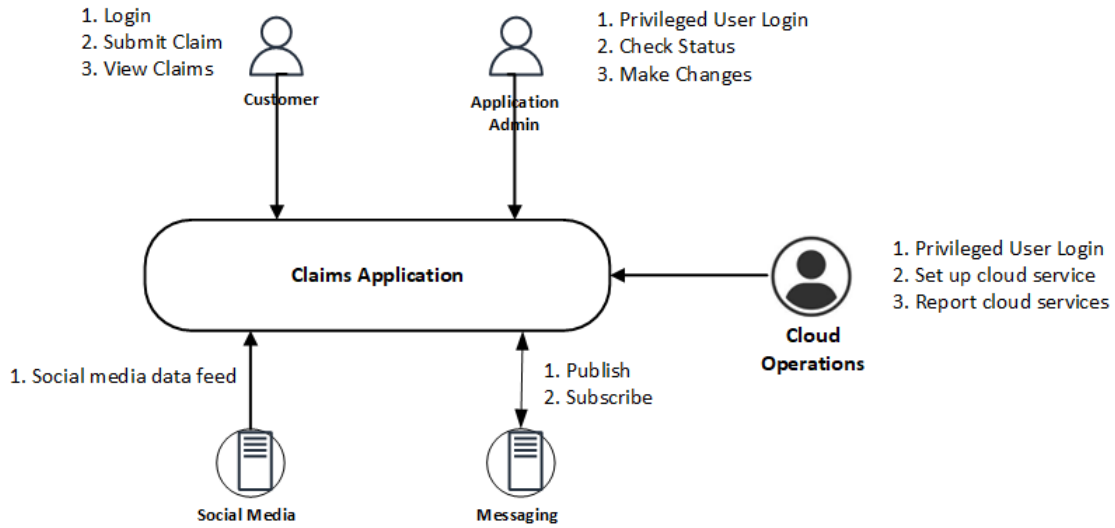
We use the System Context diagram to illustrate and describe the system boundary and the top layer of the architecture decomposition. The internal mechanisms are hidden and the system is presented with human and system actors interacting with it.

A System Context diagram helps us define the boundaries of the system and clarifies the high-level scope of the architecture work.

The System Context diagram has the following attributes:

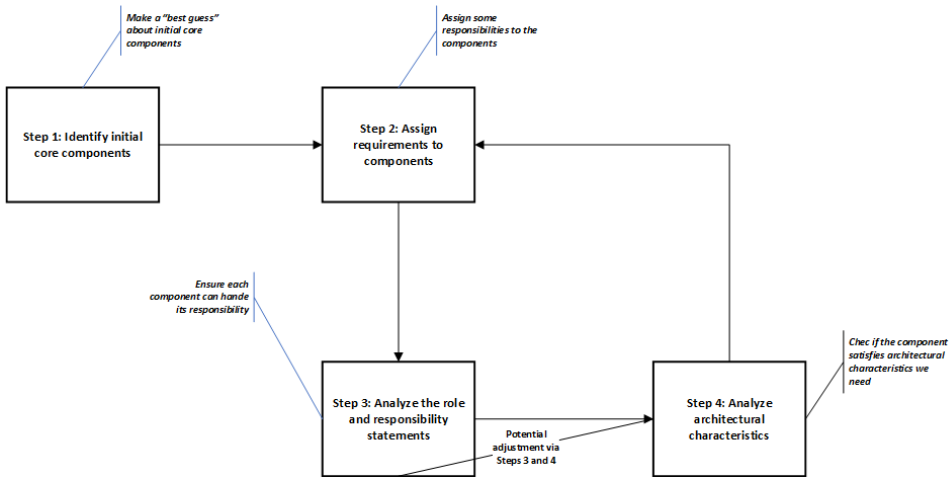
- **Human Actor**
  Individuals or groups of people that interact with the system through a set of processes or use cases where data flows in and out of the system. It includes people both internal and external to the Canada Life.
- **System Actor**
  IT systems that are external to the system. These systems are external to the system and some of them are also external to the organization For example, these systems can be owned and operated by the Canada Life or third-party (e.g., SaaS).

  It is also important not to get too detailed, with this diagram.
- **Use Case**
  For each actor, there is a list of use cases that will trigger transaction (process) flows. The transaction flow enables you to identify the kinds of data flowing to and from the system.



Component Architecture

Component Architecture diagrams are about functional building blocks of an application.



Component Architecture view has two iterations:

- Component Architecture (Solution Sketch)
- Component Architecture (Full View)

Step 1: Component Architecture (Solution Sketch)

The Solution Sketch involves identification of the initial core components of the system. This is usually an initial guessing exercise that will overtime be developed into a complete logical architecture. When naming a component, be sure to provide a descriptive name to clearly identify what the component does.

There are two common approaches for identifying initial solution sketch:

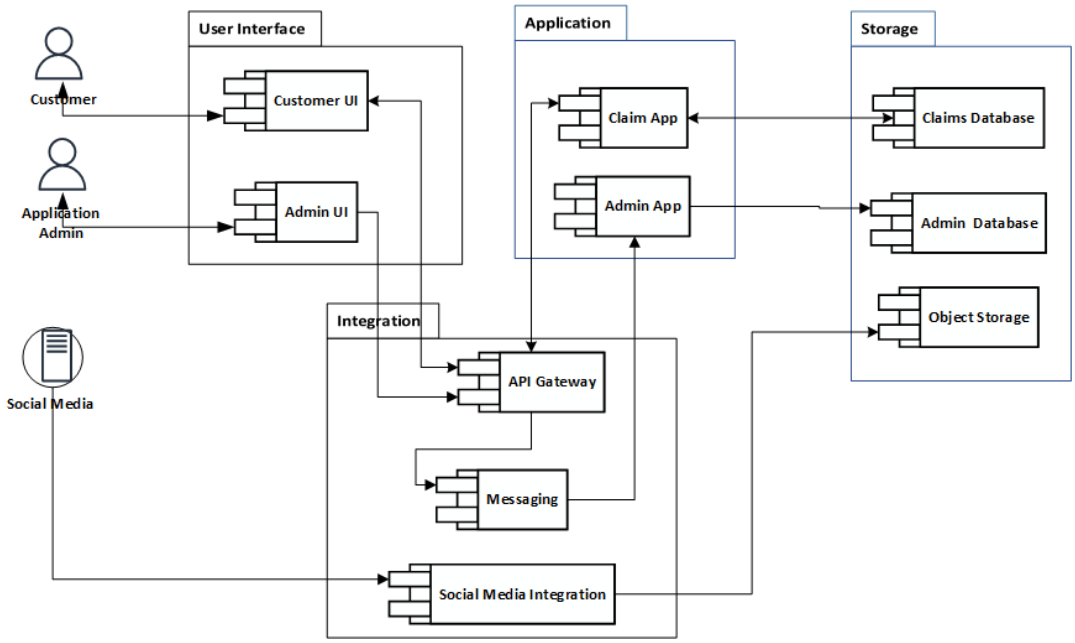
- the workflow approach and
- the actor/action approach.

They are usually used together.

Workflow approach

The workflow approach helps identify an initial set of core components by thinking about the journeys users might take through the system. For the solution sketch, do not think too much about details and concentrate on main processing steps. You can use a simplified sequence diagram which you can later use to expand the flow with more details in order to finalize the Logical Architecture View. Don't worry about capturing every step; start out with the major processing steps, and then work your way down to more details. The collaboration diagram can be also used to illustrate, on a higher level, interactions between the main components of the system. Collaboration diagrams are useful for illustrating simpler interactions between a small number of systems/sub-systems/objects and may be a better representation for a business stakeholder. If the number of objects and interactions increases, the diagram becomes hard to read. A sequence diagram might then be a better choice.

An initial solution sketch could look like this:

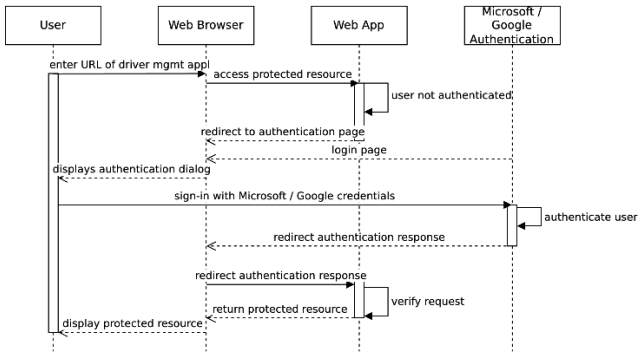


You can use the following diagram elements:

- **Component name**
  We use rectangles with the component names to depict the actual components.
- **Subsystem**
  To subsystems are illustrated by rectangles around components with the subsystems' names. Lines and arrows between components specify the relationship between two or more components. The direction of the arrow specifies the direction of the interaction between the components.
- **Actors**
  The actors in the component architecture should match up with the actors in the system context diagram.

The component architecture diagram is one of the most used diagrams outlining the components of a system.

This is a sample of sequence diagram:

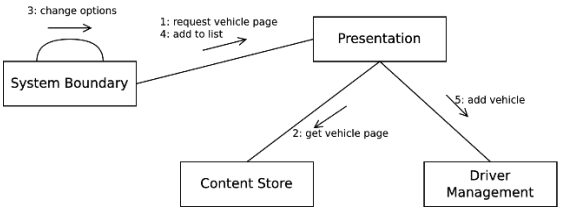


We use the following diagram elements:

- **Lifeline elements**
  Boxes at the top of the diagram represent actors and components relevant to the diagram.
- **Activation bars**
  Activation bars visualize the period of active processing of a method.
- **Messages**
  Messages represent the interaction between the actors and components. Arrows between the lifelines represent messages. A full line represents the call, and a dotted line represents the reply message, if applicable. Return messages are optional and their usage depends on the level of required detail.
- **Self-messages**
  Self-messages illustrate that a return message depends on a method execution before that return message. In the example in Figure 6-4, the authenticate user self message with the Microsoft / Google authentication illustrates that the user first needs authentication with Microsoft / Google before the return of the redirection response.

Messages are drawn from the top to the bottom to document the timely sequence of the interactions.

This is a sample of the collaboration diagram:



We use the following key diagram elements:

- **Component name**
  The actual components are depicted as rectangles with the component names.
- **Links**
  Links (solid lines) represent a relationship between the components.
- **Arrows**
  Arrows between the components represent messages. The associated numbers indicate the order of the message flow.

Actor/action approach

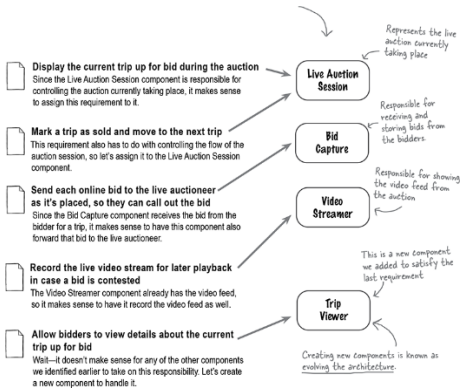
The actor/action approach is useful if we have multiple actors (users) in the system. we start by identifying the actors and then, identify some of the primary actions they might take and assign each action to a new or existing component.

Step 2: Component Architecture (Full View)

In this step, we work towards the finalization of the Component Architecture View, starting with the already created solution sketch.

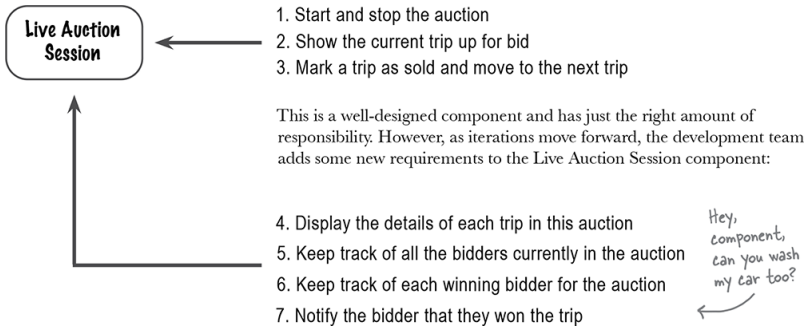
In order to finalize the Logical Architecture View, we will

- **Assign requirements to logical components** – use requirements in a form of user stories to figure out which component(s) should be responsible for each user story.



- **Analyze roles and responsibilities** - as you start assigning functionality to logical components via user stories, the roles and responsibilities of each component will start to naturally grow. The purpose of this step is to finalize the list of all functionalities that will be assigned to each component. You should also make sure that you reasonably size each component since they will be implemented as microservices in most cases.

  At the same time, make sure that functionality of component's functions are closely related.



- **Analyze architectural characteristics of the components** - the final step in identifying components of the Logical Architecture View is to verify that each component aligns with the key architectural characteristics related to but not only limited to scalability, elasticity, availability, operability, maintainability, etc.. Some components can be also merged if their functionalities are tightly coupled in order to achieve data consistency, resilience, and scalability.

Final Component View could look like this:

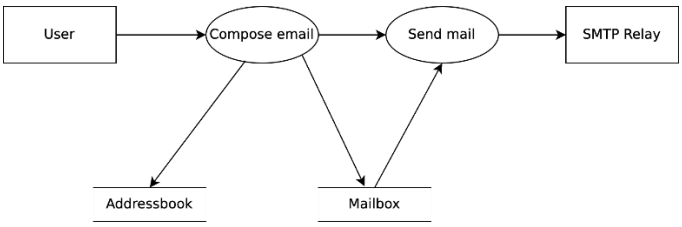
Data Flow Diagram

The System Context and Logical (Component) Architecture View concentrates on the

- systems structure,
- interaction with other internal and external systems, and
- system's decomposition into functional components and how those components interact with one another.

Another important aspect to consider is how data flows, where it's processed, and where it's stored. The data flow diagram (DFD) is a graphical representation of the data flow within a system. It depicts how

data flows from one component or entity to another within a system, emphasizing inputs, outputs, processes, and data storage.



A data flow diagram typically includes the following elements:

- **Processes (workflows)**
  Processes manipulate or transform data. They can take the form of computations, alterations to data, or anything else that's carried out inside the system. An ellipse represents them.
- **Data Flows**
  Data flows show the movement of data between system components and/or processes. They're represented as an arrow pointing in the direction of data movement.
- **Data Stores**
  Data stores are repositories or storage locations for data within the system (e.g., databases, files, or any other form of persistent data storage). They're drawn in the shape of a rectangle with parallel lines.
- **External Entities**
  External entities are entities that interact with the system from outside of it. They can be users, other systems, or any other external data source or destination. They're visualized as a box.

Data Flow diagrams are useful for system analysis and design because they depict the processing and sharing of data within a system.

Conceptual Data Model

…

Infrastructure Architecture

Component Architecture is about functional building blocks of an application.

In this step, we will develop Infrastructure Architecture which specifies where to deploy each functional component and on which type of "node."

Cloud service providers have their own notation for describing their cloud architecture and resources. We need a technology agnostic approach to describing a hybrid cloud environment including on-premises data centers and multiple cloud providers.

We use the technology-agnostic term "node" to describe the compute platform. In a cloud context, we refer to "nodes" as resource instances.

In a hybrid cloud solution, we need the following artifacts:

- **Deployment Architecture Diagram (DAD)**

We use this diagram to document the infrastructure elements in an integrated format that's cloud-agnostic for both on-premises and cloud environments.

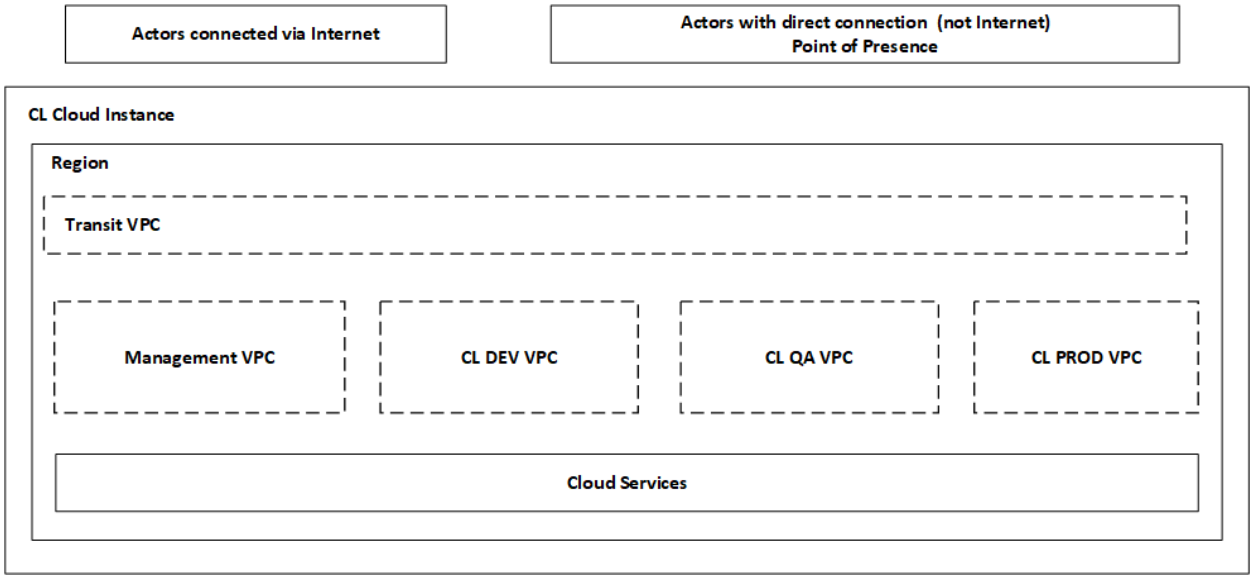
- **Cloud Architecture Diagram (CAD)**
  We use this diagram to document deployment view with specifics related to cloud providers. This diagram will define the cloud provider elements needed, the placement, the connectivity requirements, and the metadata needed for the deployment of the cloud components.

We should draw the deployment architecture diagram first as a logical diagram in a technology-agnostic way without sizing specifications. For the build and deployment of the solution, we will need a more detailed design with the specified technology and its key sizing attributes.

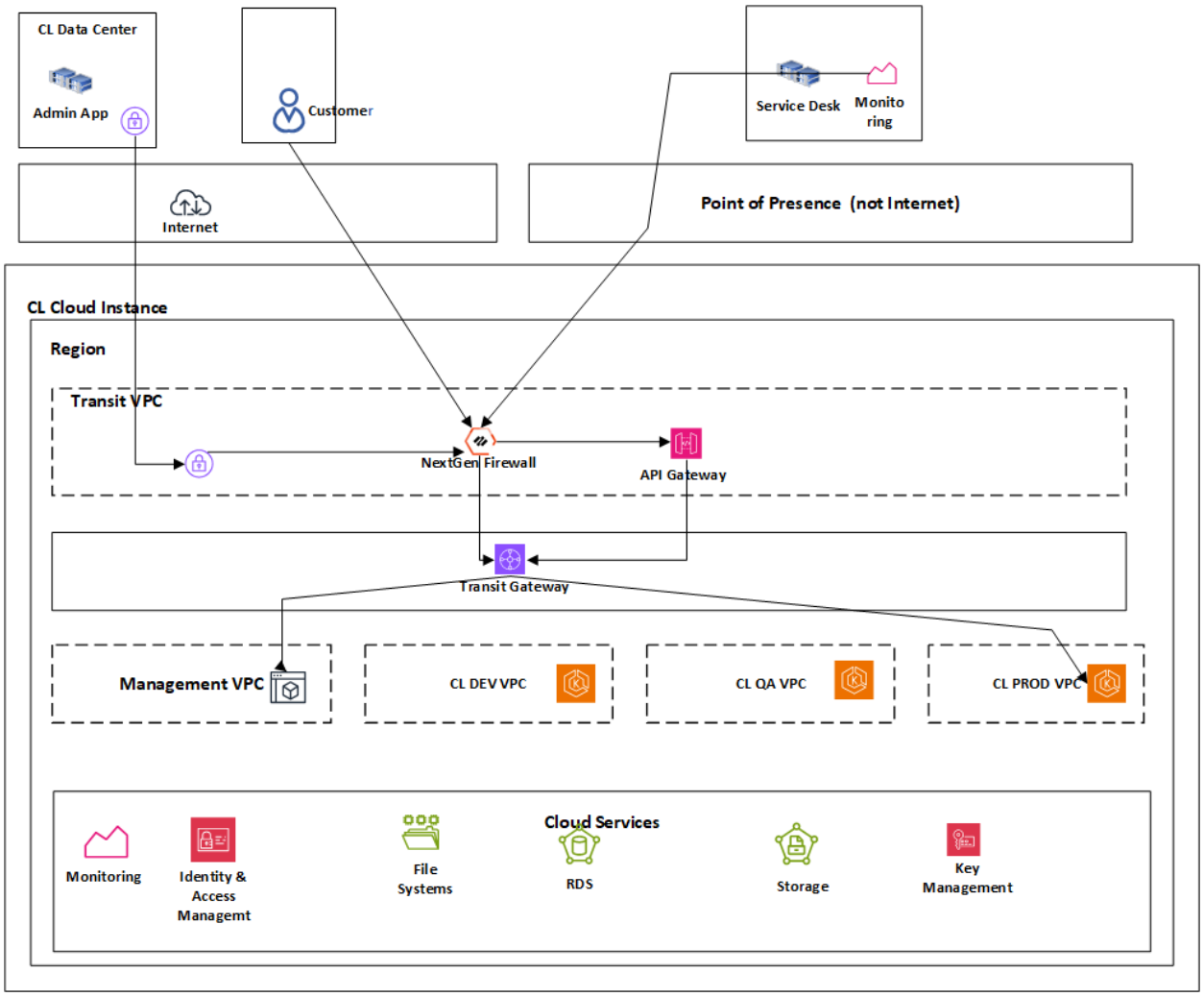
Deployment Architecture

We use a Deployment Architecture Diagram to map functional components, contained in a Component Architecture Diagram, to the nodes. The diagram can describe both on-premises and cloud locations.

The following diagram is an example deployment architecture diagram containing infrastructure elements for both on-premises and clouds. It uses a dashed rectangle to represent the network zones. The icons on black backgrounds represent the nodes with information on the selected technology and an indication of size where needed. The arrows illustrate connectivity paths between the nodes.



We use the following notation for the Deployment Architecture Diagrams:



Cloud Architecture

A Cloud Architecture Diagram is normally based on the Deployment Architecture Diagram but with additional details (if needed) and, specifics and format of the chosen cloud provider.

It usually contains additional details regarding the distribution of application components (services) and cloud services across the cloud infrastructure.

There are three different levels for setting up cloud infrastructure security:

- **Enterprise Level**
  At the enterprise level, we should have a governance model to manage configuration of both public and private cloud. This model includes but it's not only limited to
  - Adding a (new) IaaS cloud instance to an existing IT environment requires an integration design and related security measures.
  - How to set up the interconnectivity between different landing zones (both cloud and data centers), but also how to secure this network path.
  - Provide guidance on the network segmentation approach required for the given cloud instance.

    - Make and document the decisions about when you should deploy which security controls at the enterprise level and the specific controls for a given landing zone.
- **Cloud Provider/Landing Zone Level**
    - Use a cloud security control framework to address threats specific to the cloud used.
    - Define how to configure the cloud resources in line with the security policies and make these pre-configured resources available for the DevOps teams.
    - Identify if the cloud-native security tools are sufficient or if you would need third-party technologies. A recurring topic in this area is security monitoring. While this is typically organized at the enterprise level, it might make sense to use specific security monitoring solutions per cloud provider and only consolidate alerts at the enterprise level.
    - Review the operational processes to keep the applications and workloads secure in the cloud. An example could be the setup of privileged access management for the cloud instance.
- **Network Setup Level**
  As the network setup continues to evolve, it's important that the overall network design is coherent and remains consistent during expansion. The organization must monitor to ensure that the growing complexity of the network doesn't create security flaws that might go unnoticed. Also, here, the security architect should perform a continuous follow-up. Creating some architectural patterns that the architects should adopt will help keep alignment.

For the same use case, cloud providers might come with different approaches and patterns. At the network level especially, there isn't a one-size-fits-all approach. A standardized approach amongst different cloud providers could be more easily achieved at the platform level (e.g., specific Kubernetes, database engine, message broker, etc.), in all the different instances (cloud and data center).

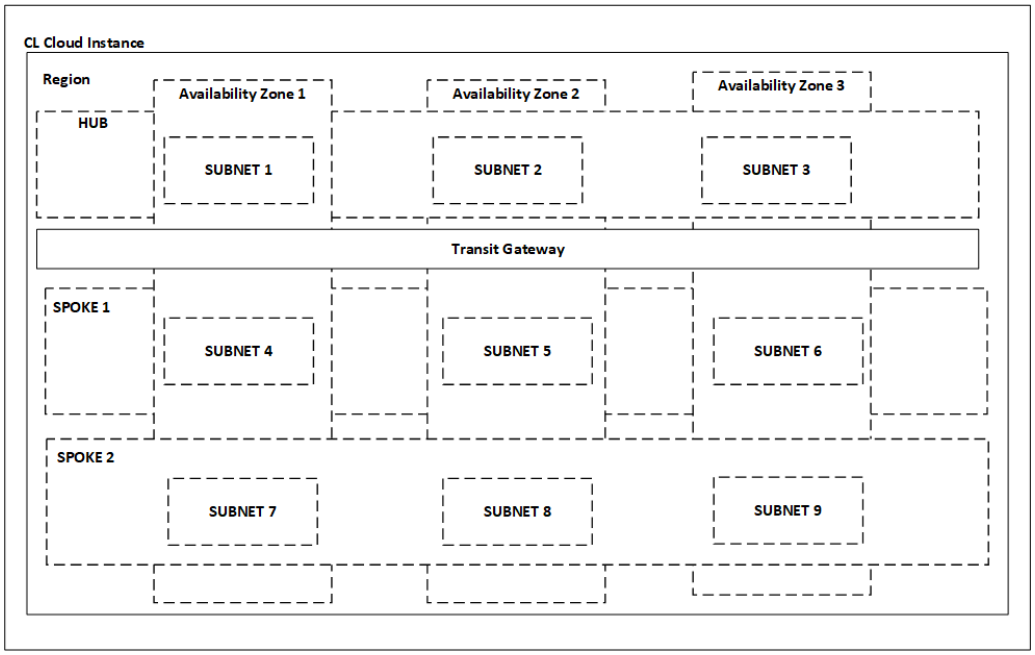
The table below list specific public cloud network design approaches and option:

Solution	**AWS**	**Azure**
Overall Network Structure	Combined model with a VPC per group of workloads	Hub-Spoke architecture
North-South Traffic & East-West Traffic	A central Ingress VPC and a central Egress VPC	All ingress & egress internet traffic goes through Hub
Separation of Production & Non-Production Environments	Most granular segmentation: Create an OU per environment and an account per application	Most granular segmentation is achieved through creation of a subscription per environment and per application; A more coarse grained network segmentation is for example the set up of a PRD subscription and a non-PRD subscription

High Availability

Most public cloud service providers, if not all, provide, per region, three or more availability zones. These zones are mostly in separate data centers in the same region with high bandwidth and low-latency. The

following diagram is a simplified Cloud Architecture diagram with availability zones but without any compote nodes, appliances, or cloud services:



The cloud architecture diagram shows a hub-and-spoke solution with VPCs spread over three zones in a multi-zone region. There needs to be the deployment of critical components over the three zones for redundancy for both the hub and the spoke VPCs.

A hub is normally called a transit or edge VPC when it comes to implementation.

If we need a web application firewall (WAF) appliance, we should deploy a WAF instance in each of the three availability zones of the Transit VPC. A load balancer should then distribute the web traffic over the three WAF instances in the different zones. In more sophisticated solutions, there will be multiple subnets per zone and per VPC.

Another view on the required availability of an application is when it’s part of the core business of the organization (so the application is part of a minimally viable company). In such a case, you must review the cyber resilience capability of the overall solution, including the supporting security solutions. How could it sustain attacks like DDoS or ransomware? How can you protect it against these threats, how will it recover from such a severe security incident where key infrastructure elements are down or data gets lost?

The analysis of the cyber resilience capabilities for a given solution is sometimes part of threat modeling, but more often this design aspect is a dedicated design activity driven by the overall business continuity strategy plan and the related governance execution.

Security Architecture

The main aspect of the Security Architecture is protecting availability, integrity, and confidentiality of the application data.

It is important to stress out that both the protection of data that application processes and protection of the infrastructure that stores and processes the data, are equally important. This is achieved by taking a ***data-centric approach*** to the design of security controls, where an architect considers how to safeguard data.

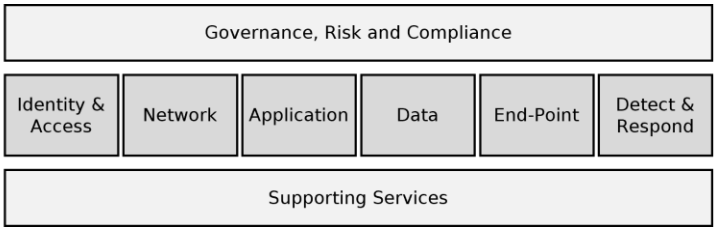
The process of designing security must emphasize data protection in transit, at rest, and in use.

Security Domains, Categories, Services

Overall, *Security Architecture* contains domains that are decomposed into categories including services:

There are six security domains that classify a comprehensive set of security services:

- Identity & Access
- Network
- Application
- Data
- End-Point
- Detect & Respond

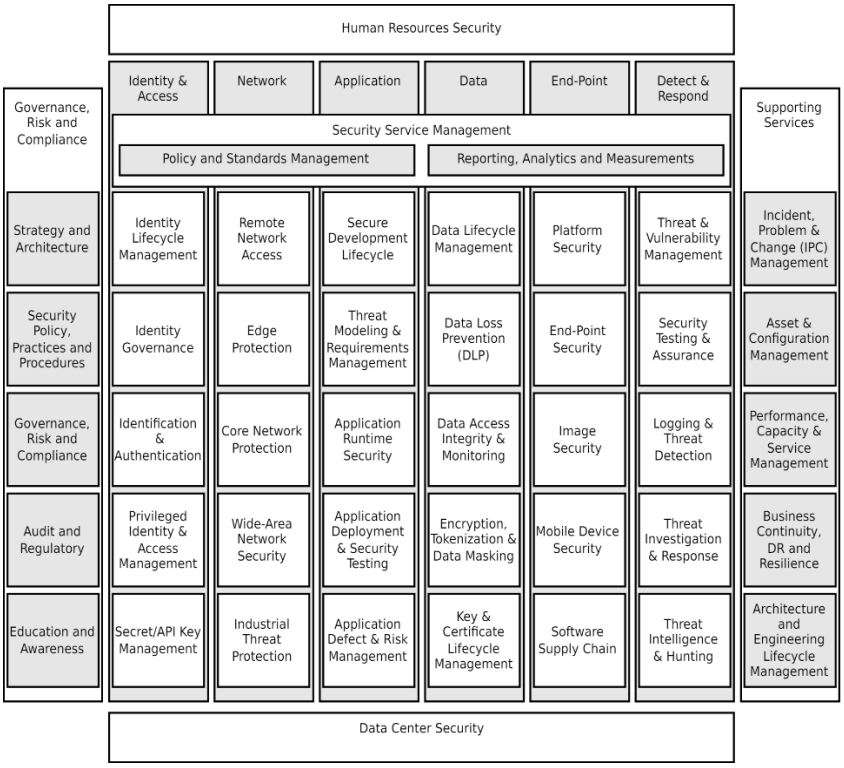


As you can notice the order of the domains is determined by a flow of granting access to an application for processing data.

An end-user will use ***Identity & Access*** before gaining access to an application. This gives access to a ***Network*** that controls access to an ***Application*** that processes ***Data*** that resides on an ***Endpoint***. Services that ***Detect and Respond*** to threats are responsible for protecting the entire application.

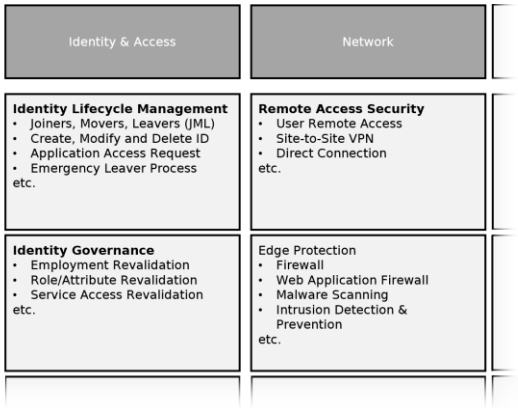
In the previous diagram, there are two additional domains also related to the *Security Architecture*. The domain at the top reflects the need for governance, risk, and compliance processes to operate the security services within the domains. The domain at the bottom provides supporting services, such as change management or capacity management, to effectively operate security services.

We decompose the security services into five categories for each domain:



The diagram becomes useful as a heat map to show what's relevant to your specific context. For example, you could shade the diagram to show what security services are relevant to the architecture you are designing. You could add colors to represent the compliance status of the service, such as red, amber, and green, enables a focus on the areas of interest.

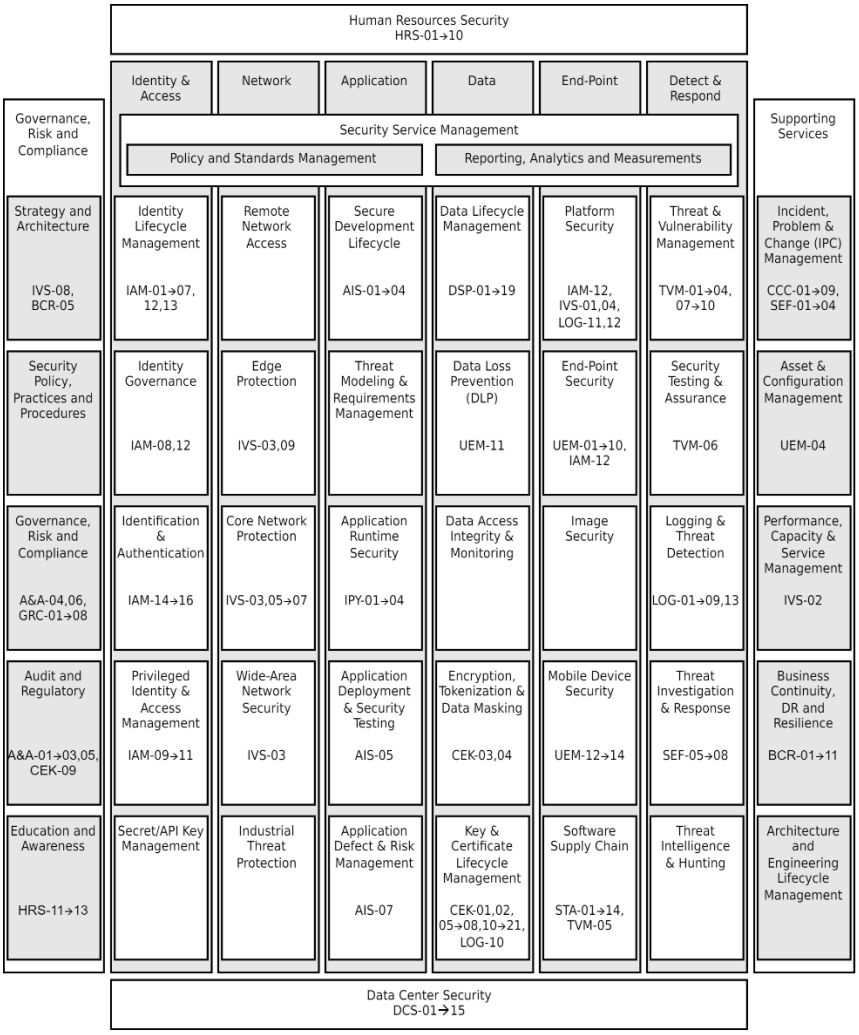
We now decompose categories into security services. For example:



In the following figure, we split the services into three layers showing the organization or team delivering the services:

	Identity & Access	Network	Application	Data	End-Point	Detect & Respond
Security Operations	• Identity Lifecycle Management • Identity Governance • Privilege Access Management	• Edge Firewall • Network Intrusion Prevention		• Hardware Security Module (HSM) • Multi-cloud key management	• XDR • Malware Detection	• SIEM • SOAR
Infrastructure Operations		• VPC Network Administration	• Application Encryption		• Secure Build • Compliance Checking	
Cloud Platform Public Services	• Cloud User Identities • RBAC • Secrets Manager	• Access Control List • Virtual End Point	• Application Encryption Toolkit	• Key Manager • Block Storage Encryption • Kafka Encryption	• Virtualisation Security • Server Encryption	• Activity Logging
Cloud Platform Private Services	• Privilege Access Manager • Privilege Access Gateway	• Trusted Time Source		• Key Manager and HSM	• End-point compliance	• SIEM

In the following diagram, we map [Cloud Security Alliance Cloud Control]() to the enterprise security architecture:



This is just an example, industry frameworks provide a good starting point but not final set of controls.

The following section contains details about how to reach a full list of security services addressing all security controls as per the requirements of the application also including governance, risk and compliance processes requirements to operate the security services within the domains.

The Component Architecture combined with Sequence Diagrams, Collaboration Diagrams, Data Flow Diagrams and Deployment View (Infrastructure Architecture) assists Application Architect and Security Architect in identifying the data that the solution or system will process:

- data in transit
- data at rest
- data generated by processing transactions (data in use)
- data classification by sensitivity and business criticality (this is just an example):
    - confidential data
    - sensitive data
    - classified or highly sensitive data
    - "Crown Jewels" - often considered business critical data or highly sensitive data

Data Centric Security

Data centric security puts a focus on analyzing the flow of data through a system from the start of a transaction to the end. At each stage of the transaction flow, we consider the security controls needed to protect data in transit, at rest, and in use.

By

- **identifying the data flowing through the system boundary,**
- **classifying it based on sensitivity, and**
- **defining the required security controls based on policy,**

we can achieve a true data centric approach to security.

The first step is to identify the data that's critical to the organization and use categories or asset classes that help identify the data that needs protection. For example, we can define the following categories:

- ***Crown jewels***
  Data that represents the most valuable information assets for the organization, and if disclosed, modified, or made unavailable, it could cause the business to fail.
- ***Personal Information (PI)***
  Personal data that if it is disclosed, it shouldn't cause irrevocable harm to the person. For example, name and address.
- ***Sensitive Personal Information (SPI)***
  Personal data that if it is disclosed, it could cause irrevocable harm to the person. For example, physical or mental health, political opinions, racial or ethnic origin, religious beliefs, etc..
- ***Financial Information***
  Personal financial data, including bank records, tax records, and accounting records.

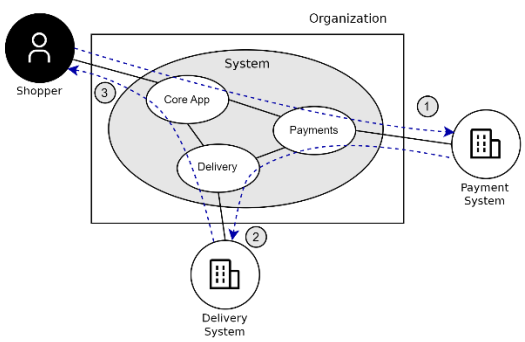
The second step is to go through the data security lifecycle to figure out security controls to protect data from unauthorized access, modification, or disclosure during *all* stages of the data lifecycle. The controls

that need design and implementation will depend on the sensitivity of the data as per above classification.

The data processed by a system, go through a lifecycle including

- **Creation**
  The initial creation of data.
- **Storage**
  Retention of data, either at rest or in processing.
- **Aggregation**
  The integration of data from multiple sources into a single dataset. The aggregation may increase the sensitivity of the resulting data.
- **Generation**
  The creation of new data through processing or analysis of existing data.
- **Transmission**
  The movement of data from one location to another.
- **Access**
  The retrieval and use of data.
- **Backup and Archive**
  The medium and long-term retention of data.
- **Destruction**
  The removal of data from a system.

The diagram below shows the flow of data as it passes through a system:



In the diagram, a shopper submits an order to the system. The box represents the organization, and the outer ellipse represents the boundary of the system processing the data. Within the system, there are three interconnected subsystems that process the data. The dotted line represents the transaction flows that transport the data through the subsystems and require protection.

The above diagram is just an oversimplified diagram. You should draw detailed data flow and sequence diagrams for this step.

Let's walk through each of the transaction flows in order:

1. The submitted order triggers a transaction flow from the shopper to the "Core App" subsystem. When the order is ready to complete, the flow continues onto the "Payment" subsystem and out to an external payment system, as shown in flow 1.

2. After completion of the payment, the transaction returns to the “Payment” subsystem and moves onto the “Delivery” subsystem to connect externally to arrange for delivery, as shown in flow 2.
3. After the arrangement of the delivery, the confirmation of the order is then returned to the “Core App” subsystem and the person who placed the order, as shown in flow 3.

At many of the steps in this transaction flow, processing and aggregation of data are taking place, increasing the value of the data and the need for increased controls. At all stages of the transaction flow, we need to consider the security controls to protect the data from loss of confidentiality, integrity, and availability.

Once we understand the transaction and data flows we can move onto Secure by Design with Threat Modeling.

Secure by Design with Threat Modeling

Secure by Design uses threat modeling as a way to identify risk-based security controls for transactions and data that move through a system.

Threat modeling identifies specific threats and will build on corporate

- security policies,
- practices, and
- processes

that don’t focus only on the specific threats to sensitive data. We extend the practice to the examination of the application and infrastructure architecture for the identification of risk-based controls.

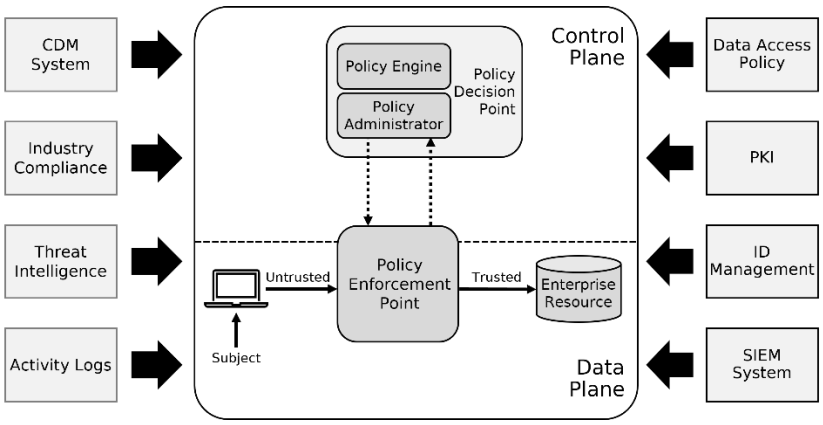
To ensure the identification of all sensitive data, we use a systematic architectural thinking approach for the examination of all the important data flows and transactions. Threat modeling practices need to work at scale across multiple computing platforms in a hybrid cloud environment.

Zero Trust Architecture

Zero trust is a model that removes implicit trust within a system boundary and continuously evaluates the risks by applying mitigations to business transactions and data flows at every step of their journey.

The zero-trust model assumes that all business transactions and data flows, whether originating from inside or outside the network, are potentially malicious. Every interaction in a business transaction or data flow must be continuously validated to ensure that only authorized users and devices can access sensitive business data. In effect, it moves the perimeter from the system boundary to the point at which identification, authentication, and authorization take place, resulting in identity becoming the new perimeter. The whole concept often gets simplified down to the **“never trust, always verify”** principle, but it’s more than that.

Zero-trust architecture requires a cultural shift that emphasizes the importance of security rather than just compliance throughout an organization. This means that implementing a zero-trust architecture involves not only the deployment of specific technologies but also the development of processes and practices that promote a data security first mindset across the organization, building on the data centric security approach we discussed earlier.



The diagram has a subject (human actor or user) using a system to access an enterprise resource (data or functional component). Between the subject and resources, a policy enforcement point (PEP) acts as a proxy determining what resource the subject can access. The PEP uses the policy decision point (PDP) to make those decisions.

A more detailed breakdown of the key elements follow:

- **Subject**
  The subject can be a human actor, system actor, or functional component accessing an enterprise resource, such as data.
- **Enterprise Resource**
  This is a resource under the control of your organization, such as data, a device, or functional component.
- **Policy Enforcement Point (PEP)**
  The PEP provides inline access control between the subject and the enterprise resource managing the connection. The PEP will forward every access request to the PDP for evaluation, and the PEP will enforce the decision returned by the PDP. In reality, the PEP (and the PDP) often run on the same system as the enterprise resource for performance reasons. There are multiple types of configurations possible.
- **Policy Decision Point (PDP)**
  The PDP contains two components. The first component is the Policy Engine (PE), which takes access decisions based on two data elements. First, it requires policies that define who gets access to what kind of resource and under what conditions. Second, the contextual data needed by the access decision rules. As you can see in the diagram, there is a wide range of diverse data sources that could provide contextual data. The second component is the Policy Administrator who manages communication with the PEPs.
- **Control and Data Plane**
  The PEP manages the traffic in the Data Plane, it's the network path carrying the application data. The communication between the PEP and the PDP is through the Control Plane which is a restricted component that's separate from the Data Plane.
- **Data Sources**
  The components left and right represent possible data sources of contextual information for the PE (all the arrows should rather point to the PDP as they represent the flow of contextual information to the PE). The contextual information could be about the subject who is requesting access, the device, the network path, or the system hosting the data. A few terms that need expanding include:

- The Continuous Diagnostics and Mitigations (CDM) system monitors the state of devices and applications and should apply patches/fixes as needed
- The Security Information and Event Management(SIEM) system detects potential threats and vulnerabilities.

This conceptual architecture is there to support the **"Never trust; always verify"** principle and **is applicable to these use cases where policy enforcement points (and the related policy decision points) can make use of contextual data to make access decisions.**

Now we'll discuss the translation of the zero trust principles into practices.

Zero trust principles to practices mapping:

Principles	**Practices**
Data-Centric Security	***Identity, Data and Transaction Identification*** We need to understand resources in order to know what and where we need to protect the data assets. We do this by identifying the actors, data, and transaction flows through the system. This is the identification flow: 1. Creation of a ***System Context*** diagram to identify users, devices, and services that are external to the system. 2. ***Application Security***: The process continues through understanding the transaction flows and the threats to the data, traveling through the functional components. 3. ***Infrastructure Security***: The process repeats with the flows through the deployed components on the infrastructure.
Never Trust, Always Verify	***Continuous Authentication*** Authentication is required not only at the start of a user or system session but also continuously during the lifetime of the session, also known as continuous authentication. If contextual information changes during the active session, then the system re-evaluates the access policies, and the outcome could be the termination of the active session. Continuous authentication may, for example, include biometric authentication such as fingerprint scans, contextual information such as time of day or location, etc.. Detection of threats is possible by using the alerts from the continuous authentication mechanism when authentication fails. Continuous authentication is used as a part of the threat modeling analysis for the transaction flows through the system (Infrastructure Security).
	Adaptive Access Control Adaptive access control provides dynamic access adjustments depending on the context in which the user is accessing the system. For example, behavioral patterns may alter the level of access depending on information, such as normal login times. Risk-based decisions may enable higher levels of access if using a trusted device and other contextual information, such as the location of the device.

	Adaptive access control should be implemented via the real-time access control components such as PEP or the access logic built in and should make use of the maximum possible information from the available contextual data. Once a system makes a decision on the level of access, the further refinement of the permissions can be achieved by a combination of access control patterns such as Role-Based Access Control, Attribute-Based Access Control and Risk-Based Access Control. Currently in most systems, adaptive access control is not in place for two reasons: impacts on performance and sometimes relative complexity of the solution. The main focus has been on zero-trust solutions at the network edge using Zero Trust Network Access (ZTNA).
	Least Privilege A user or system should receive the minimum permissions to complete the activities they need to perform. For sensitive processes, there should be the enforcement of separation of duties. Fine-grain access controls, needed to achieve the least privilege implementation, can be designed by using the Role-Based Access Control, Attribute-Based Access Control and Risk-Based Access Control patterns.
	Micro-Segmentation Today, the micro-segmentation is used, where compute resources are in small, isolated segments to reduce lateral movement on a network (Infrastructure Security)
Data Protection Everywhere	***Encryption in Transit, At Rest and In Use*** Encryption of data is an essential mechanism to protect sensitive data from disclosure, particularly on a cloud platform where we are not in control of cloud operations. Encryption of data at rest needs to effectively prevent privileged users from having access to the most sensitive data (Infrastructure Security).
Assume Breach	**Threat Detection and Response** With the assume breach dimension, the focus shifts to security monitoring to detect possible anomalies in all security domains. During the threat modeling process that is involved in the *Application Security* and *Infrastructure Security*, you should discuss the identification of potential threats and any abnormal behavior. You can then use the identified threats for threat detection and incident response.

Some examples of zero trust based solutions:

Zero Trust Principle	IAM	Data	Application	End Point	Network
Never Trust, Always Verify	• Just in Time access • Privileged Access • Continuous Authentication • Adaptive Access Control	• Adaptive Access Control	• Adaptive Access Control • Allowed Processes List	…	…
Data Protection Everywhere		• mTLS • Certificate Manager • Key Manager & HSM	• mTLS • Certificate Manager • Key Manager & HSM	…	…
…	…	…	…	…	…

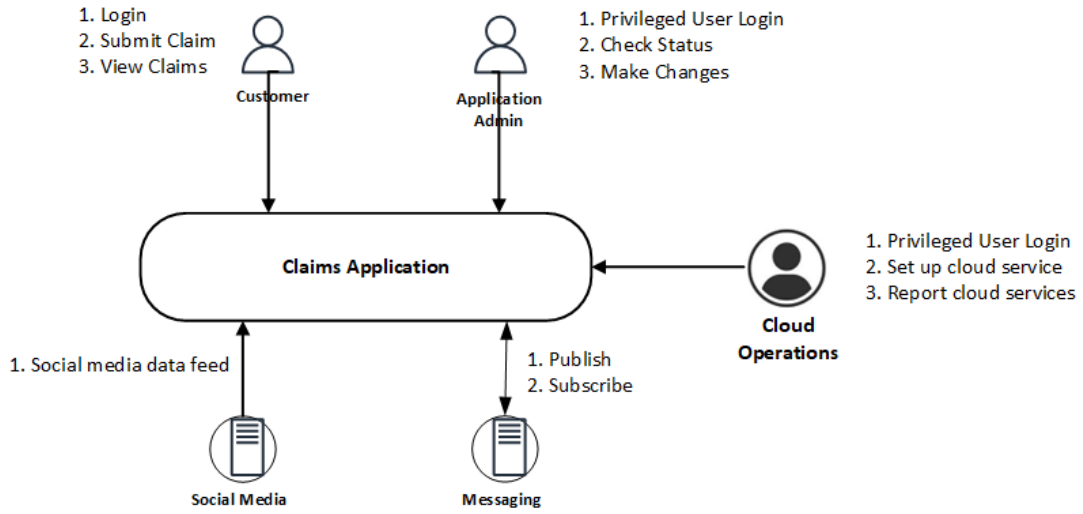
Application Security – Use Case (Example)

The development of Application Security has three parts:

- System Context
- Component Architecture
- Threat Modeling

System Context

We start with the System Context diagram:



We've got three types of actors in our system:

- Functional human actors, illustrated as solid user icons. Those actors will need a user interface to interact with the system.

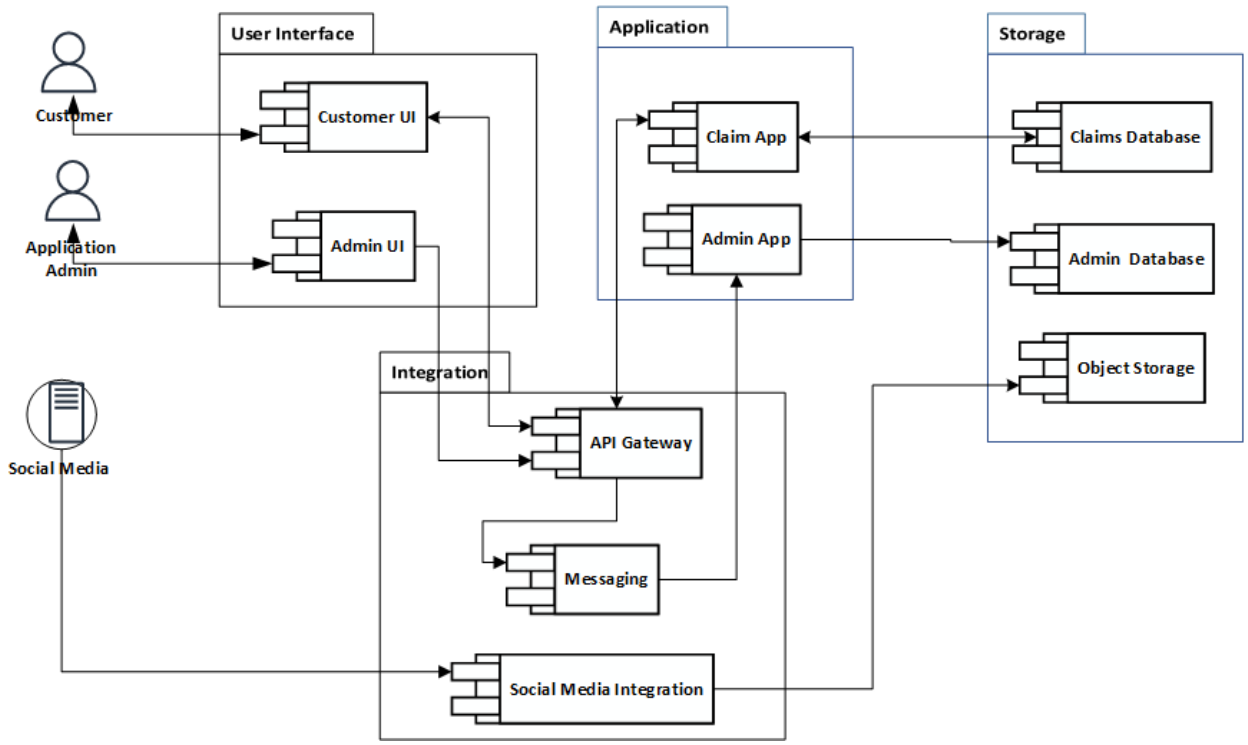
- Operational human actors, illustrated as outlined user icons. Until we've established the operational view of the solution, we wait to include those actors in the decomposition of the system.
- System actors, illustrated as building icons. Those actors require integration capability to either receive incoming functional calls or establish outgoing functional calls.

Component Architecture

The next step is to create Component Architecture View. In order to do this, we have to figure out functional building blocks (components) of the system. The table below contains the components:

Type	**Purpose**	**Technology**	**Component**
Application Components	Claim app	Java 21, Spring Boot	AC-01
	Admin App	Java 17, SpringBoot	AC-02
User Interface	Customer UI	React.js	UI-01
	Admin UI	React.js	UI-02
Databases & Storage	Claims Database	MongoDB Atlas	DBS-01
	Admin Database	AWS RDS	DBS-02
	Object Storage	S3	DBS-03
Integrations	Messaging	Amazon MSK	INT-01
	Social Media Integration	RESTful API	INT-02
	API Gateway	Apigee	INT-03

Based on this information, we can now draw the initial version of the component architecture (Solution Sketch) for our system:



This component architecture forms the basis for the next step which is about modeling threats and countermeasures for our system.

As mentioned earlier, we've not yet considered the operational actors in our component architecture, as they mainly represent non-functional aspects of the solution. Once we've developed the deployment architecture of our system, we might need to adapt the component architecture.

In the checklist, we provide you with important considerations to remember when you develop the component architecture.

COMPONENT ARCHITECTURE QA CHECKLIST

- Make sure that you represent all functional actors in the component architecture and understand what interface they use to interact with the functional components.
- Verify that you have defined components to satisfy all identified functional requirements.
- Verify that you have identified all interfaces between the components and that you understand whether the interface is passing or receiving the respective data. Is the data that you identified during the development of the system context diagram flowing correctly through the functional components?
- Because you don't know everything yet, you've made several assumptions throughout the process. Make sure you have documented those assumptions in the RAID log.
- Application architects who develop the component architecture will have to think about much more details, which aren't that relevant for what we, as security architects, are using the component architecture for.
    - Verify that you partitioned the components in a way that allows the development work to be divided between developers.

    - Verify that you loosely couple the components to allow rather independent development work

Threat Modeling

Now we will work on the Threat Modeling.

Security requirements derive from

- external rules,
- internal policies and standards, and
- the actual threats exposing a system.

We need to figure out how to breach the system to identify countermeasures to detect, correct, and prevent threats.

Threat modeling is a process to identify those threats to the system, the associated risks, and the correct controls to produce effective countermeasures.

Threat modeling is the key element of application security and the secure by design discipline. When done correctly, threat models can help teams identify and mitigate issues early in the solution life cycle. Threat models also help feed downstream security processes like assurance processes (for example, QA testing, code scanning, and penetration testing) as well as threat detection use cases.

Threat modeling is an open never ending exercise. The level of creativity, expertise, and time available limits the number of threats you are able to identify. There are various threat modeling techniques available that require different levels of expertise and effort with varying numbers of layers. The following table lists some of these techniques.

le 6-2. Threat Modeling Techniques

evel of xpertise	Threat Model Layers	Rooted in		
		Systems Engineering	System Analysis	Process Engineering
igh	• CVE • CWE • Security Controls • Persona non-grata • MITRE ATT&CK framework	TARA	• PASTA • Attack Trees	
edium	• Application Components • Infrastructure Components • Data Flows • Threat Actors • Threat Vectors	Trike	OWASP Application Threat Modeling	Microsoft Threat Modeling Tool
ow	• Boundaries/Zones of control • Assets • Threats • Threat Actors • Controls/Countermeasures	STRIDE		VAST

We're describing here a common approach for threat modeling that allows you to apply different threat modeling techniques depending on the type of system you are assessing and the level of expertise of the people involved in the threat modeling process. This assumes a low level of expertise in threat modeling.

For this approach, it's required to have a "visual" of the solution design. A commonly used artifacts are component architecture, sequence diagrams and data flow diagrams. We recommend selecting key use cases (process flows) and following the data through the solution.

For simple architectures, we can use tags within the visual to mark the elements of the threat model. For more complex architectures, it's good practice to use threat modeling tools to support this process.

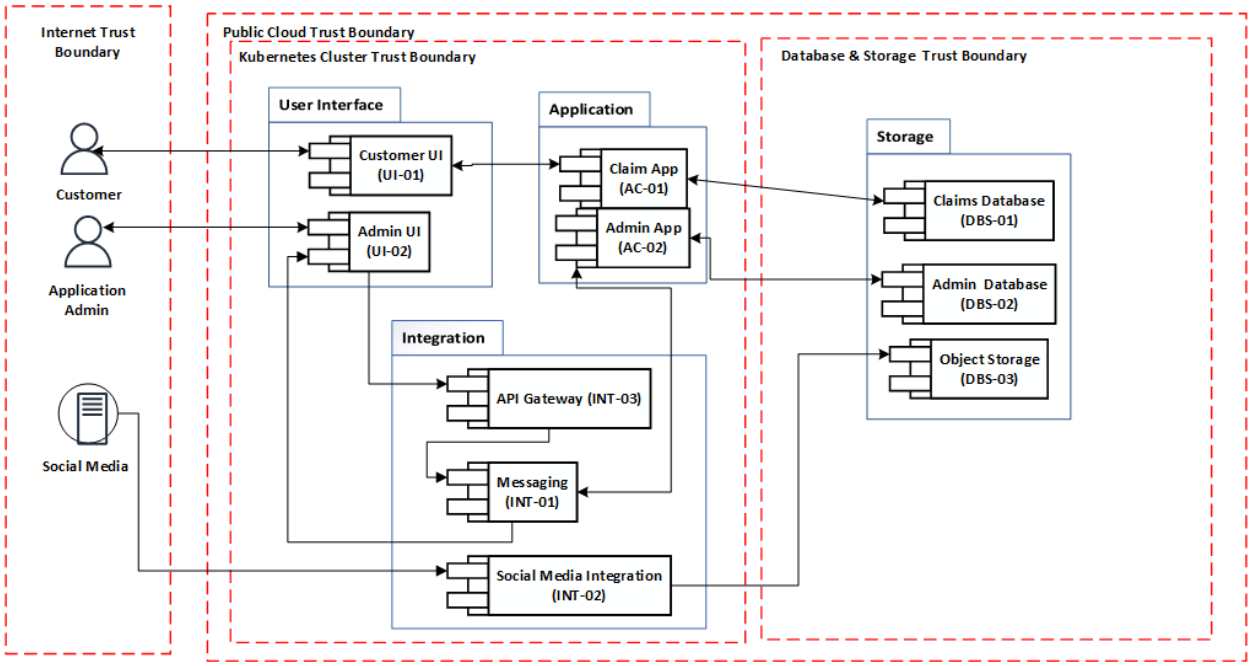
There are six steps in the development of the threat model:

- Identify boundaries
- Identify Assets
- Identify Threat Actors
- Identify Threats
- Identify Controls
- Prioritization of Controls

Identify boundaries

Boundaries are demarcations that separate one entity from another and where data flows from one entity to another. These boundaries represent the attack surface of the system, and many threats will cluster around these boundaries. The trust boundaries are a point of security enforcement where you place firewalls, enforce authentication, terminate encryption, etc. It's important to know where the trust boundaries are within the architecture (system, solution, product, etc.).

A combination of the component architecture and the deployment architecture will provide the most comprehensive basis to identify and draw the trust boundaries. For example, the following diagram illustrates trust boundaries:

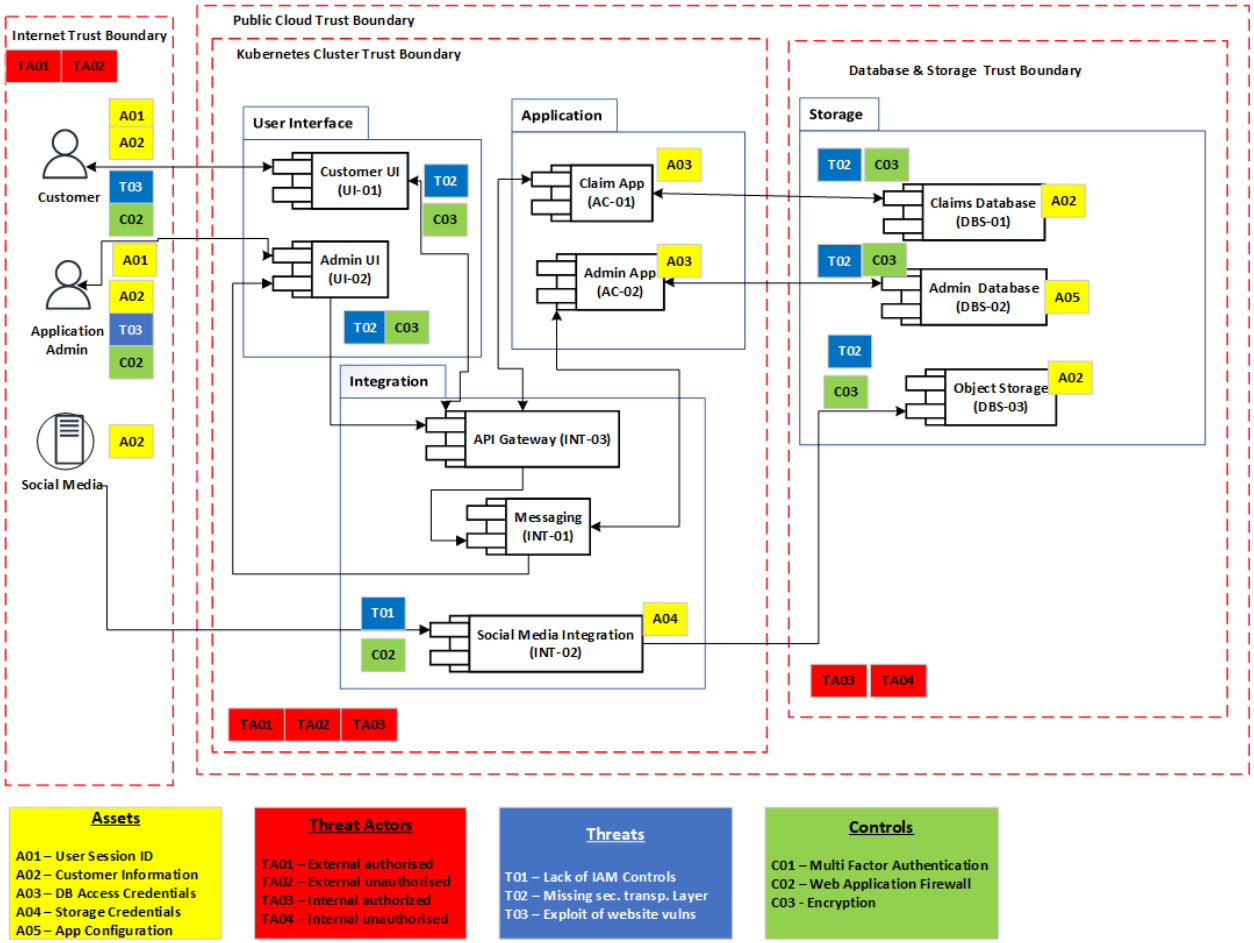


Identify Assets

We need to identify assets that need protection in the system. We also need to list their locations. Please see the data classification example in the *Data Centric Security* section.

Storage of data takes place in different locations: data stores, credential stores, 3rd party tools, solutions, and software.

The following diagram is an example of the visual illustration of identified assets on the component diagram:



You should also document the type and volume of assets identified.

Identify Threat Actors

At this point we know where the assets are and what threats they're exposed to within the system. Now we need to understand who the threat actors are. By knowing the actors, we'll properly understand the likelihood of the exploitation of a threat by the threat actors in relation to the location of the threat in the system.

On a high level, there are two categories of threat actors:

- **External Threat Actors**
  External threat actors are external to the organization. There are two types of them:
  - External authorized malicious actors – individuals with legitimate access and malicious intent.
  - External unauthorized malicious actor – individuals without legitimate access and malicious intent. This can also be further broken down into nation-state-sponsored, activists, criminal organizations, etc.

- **Internal Threat Actors**

Internal threat actors are internal to the organization. This can be an employee, a contractor, etc. someone who has legitimate insights about the organization. There are three types of them:

- Internal unauthorized malicious actor – individuals without legitimate access and with malicious intent
- Internal authorized, "honest but curious" actors – individuals with legitimate access to the data, just not all the data.
- Inadvertent - These individuals inadvertently compromise security through careless actions or a lack of awareness.

Please see the list of threat actors on the previous diagram

Document the types of threat actors.

The above proposed classification of threat actors might not be ideal. It can be updated in order to come up with the most relevant threat actors list for our organization.

Identify Threats

In case of manual threat modeling, we can start with high-level threats informed by OWASP Top 10.

For a more structured approach to developing threats and mitigation techniques, methods like STRIDE or attack trees, are recommended. More details about these techniques are available in Threat Modeling - Designing for Security.

STRIDE

STRIDE is a threat modeling method that assists in the identification and categorization of potential threats in software systems. It offers a systematic way to identify and mitigate security risks. Each letter of the word STRIDE represents a different category of threats:

- **Spoofing**
  An attacker impersonates another user, entity, or system to gain access to the system.
- **Tampering**
  Attackers may modify data, change system settings, or install malicious code.
- **Repudiation**
  An attacker denies performing an action by, for example, modifying logs, interfering with audit trails, or fabricating records.
- **Information Disclosure**
  Sensitive or secret information is inadvertently or maliciously given to unauthorized parties. It can be caused by vulnerabilities that expose data during transmission, processing, or storage.
- **Denial of Service (DoS)**
  Denial of Service (DoS) threats try to disrupt or prevent the normal operation of a system or service. Attackers may flood the system with requests, etc.
- **Elevation of Privilege**
  Attackers gain illegal access or privileges within a system. They may use vulnerabilities to increase their privileges, assign specific access controls to themselves, or do operations that exceed their authorized degree of access.

You should examine the system's components, relationships, and capabilities to determine how each threat category might materialize. The goal is not the categorization of threats, but the identification of

them. Microsoft developed a card game Elevation of Privilege (EoP) to support the creative process of identifying threats. This game helps teams to think about how to break the solution.

Attack trees

Attack trees are graphical models used to represent and analyze potential attack paths and scenarios in a system. They provide a structured way to understand the steps an attacker might take to exploit vulnerabilities and achieve specific goals.

The MITRE ATT&CK framework provides a framework for a more low-level threat analysis as well as for the identification of mitigation approaches. We still recommend to leverage threat modeling tools that feed information from MITRE, NIST, and CERT databases and allow a more comprehensive way of documenting the different elements of a threat model.

Please see the identified threats on the previous components diagram with threats.

Document the types of threats.

LINDDUN

Privacy related issues are not specifically addressed in the previously mentioned techniques. The LINDDUN Privacy Threat Modelling Framework addresses the privacy related issues. LINDDUN specifies seven privacy threat types:

- **Linking**
  Linking data items or user actions may have unintended privacy consequences.
- **Identifying**
  Many systems require the identification of data subjects; identifying threats arise when leaks, deduction, or inference reveal the identity, even if it is not intentional or desired.
- **Non repudiation**
  Being able to assign a claim, i.e. knowing, doing, or saying something, to a specific person.
- **Detecting**
  Observing someone and determining their level of involvement. Detecting threats does not require the ability to read the actual data; simply knowing that it exists is enough to infer more (sensitive) information.
- **Data disclosure**
  Excessive collection, storage, processing, or sharing of personal data.
- **Unawareness and unintervenability**
  Inadequate information, involvement, or empowerment of individuals in the processing of their personal data.
- **Non compliance**
  The system deviates from best practices in security and data management, as well as standards and legislation, resulting in incomplete risk management.

Identify Controls

Now when we have identified treats, we must define the controls that will mitigate them.

There are three categories of security controls:

- detective,

- preventive, and
- corrective.

Each category focuses on a different aspect of security and aims to mitigate risks in distinct ways. Here are the definitions of each type of control:

Detective Security Controls

Detective controls identify and detect security incidents or violations that have already occurred. Detective controls include:

- **Intrusion Detection Systems (IDS)**: These systems monitor network traffic, looking for patterns or signatures that indicate potential security breaches or malicious activity.
- **Security Information and Event Management (SIEM)**: Collects and analyzes logs and events from different sources in order to discover potential security incidents and provide real-time alerts.
- **Log Monitoring and Analysis**: Revies and analyze logs to detect unauthorized access attempts or suspicious activities.
- **Security Audits and Assessments**: Applications, systems, and networks have to be assessed and audited to identify potential vulnerabilities, weaknesses, or non-compliance with security policies.
- **Continuous Compliance**: Continuously monitor organization's information technology assets to ensure that they're in compliance with defined security standards.

Preventive Security Controls

Focus on preventing threats from gaining access to sensitive information. Preventive controls include:

- **Access Control Systems**: Provide authentication and authorization services, ensuring that only authorized users can access application, data, and services.
- **Firewalls**: Filer network traffic between internal and external networks and block unauthorized access attempts.
- **Antivirus and anti-malware software**: Scan for viruses and other fors of malware, and prevent execution of malicious software.
- **Security Awareness Training**: Security best practices and policies to prevent social engineering attacks and improves overall security. Train employees and educate customers.
- **Software Development Proactive Controls**: OWASP has a list of the top 10 security techniques in the OWASP Proactive Controls, which should be included in every software development project.

Corrective Security Controls

Measures taken after the identification of a security incident or breach. They aim to mitigate the impact, recover from the incident, and restore normal operations. Corrective controls include:

- **Incident Response Plans**: Specify the actions and procedures that organizations must take in the event of a security incident to ensure a coordinated and effective response.
- **Backup and Disaster Recovery**: We must have regular back up of data and a well-defined and tested disaster recovery plan to help recover from incidents and minimize data loss.
- **Patch Management**: Regularly apply security patches and updates to systems and software.
- **Forensic Analysis**: Conducting forensic analysis and investigation helps determine the cause, extent, and impact of a security incident and gathers evidence for legal purposes.

Preventive controls have, in many cases, the biggest effect as they stop bad things from happening, but they usually come with the highest cost. Detective controls are the second choice, as they help to decrease the time to respond to an incident and, with that, the impact on the organization. And the last resort are corrective controls.

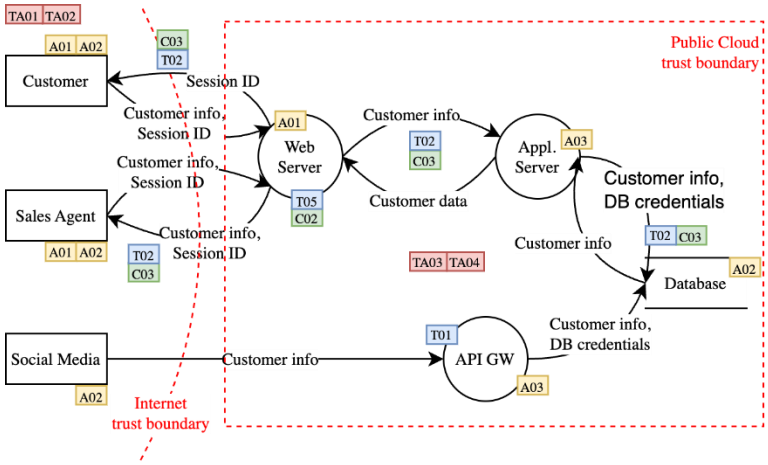
We might also face situations where the cost of the protective control outweighs the benefit, and a detective control might be the better choice.

The following frameworks can be used for identification of relevant and effective controls:

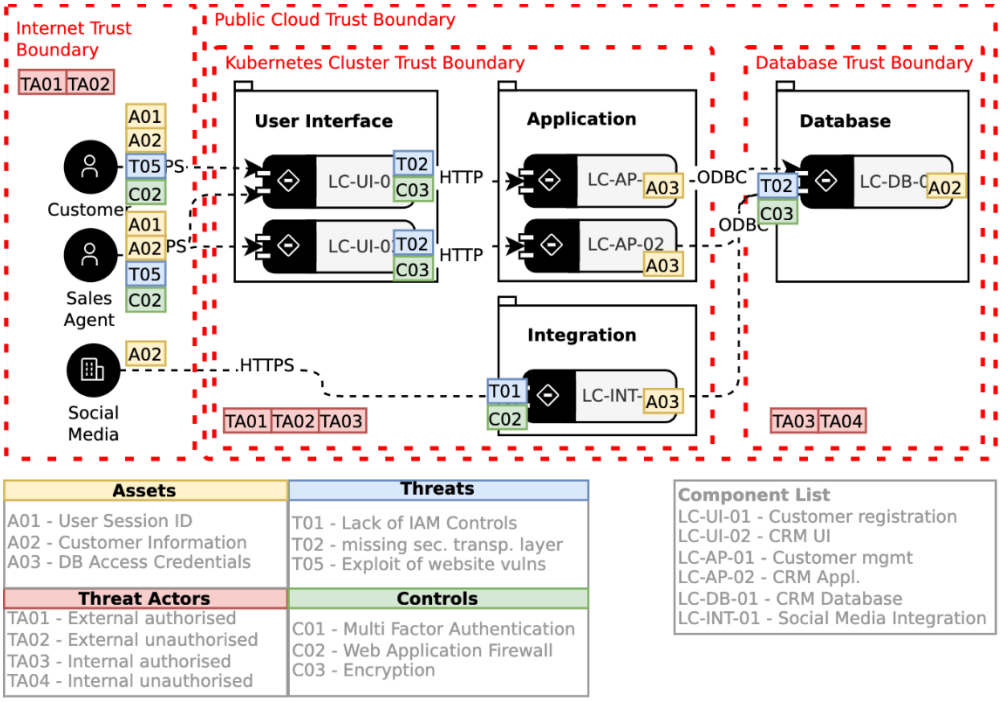
- MITRE ATT&CK framework
- Common Attack Pattern Enumeration and Classification
- The 18 CIS Critical Security Controls

It is important to consider, when we identify threats and effective controls, that assets don't only exist at rest in our solution but also in motion from one component to another.

As we mentioned earlier, this threat modelling process can either be based on a component architecture diagram or a data flow diagram. The following diagram visualizes the threats analysis via the data flow diagram:



Adding all the information from the last five steps into one diagram



Prioritization of Controls

At this point we have a list of controls that are mitigating all the identified threats in the system. However, some identified controls might be complex and costly to implement and maintain. If, for example, the cost of implementing a security control is higher than the cost caused by exploiting the associated threat, then it doesn't make to invest in that control from the business perspective.

That means we need to understand the risk the threat is posing to the organization. Here is where our Risk Framework is also getting utilized.

The common definition of risk is: Risk = Likelihood x Impact. Within cybersecurity, qualitative risk analysis is the most commonly used approach to evaluating risk, which we're also applying in this book. Though the adoption of quantitative risk evaluation methods like the FAIR method or ISF Quantitative Information Risk Assessment (QIRA) has increased in recent years. The drawback of qualitative methods is that there is bias based on the experience of the individual evaluating the risk. While it might be easier to identify the impact category of a risk, it's difficult to consistently identify the likelihood of a threat. To reduce the impact of the experiences of individuals, OWASP and other organizations have developed risk rating methods like the OWASP Risk Rating Method with its associated calculator. Using this method removes a lot of bias and produces good enough, reproducible results.

The OWASP Risk Rating Method divides likelihood into threat agent and vulnerability factors, and impact into technical and business impact factors. We can influence all factors except the threat agent factor through the chosen risk treatment action. Threat agents are external factors outside of our area of influence.

Threat Modeling Tools

In the previous section we demonstrated manual threat modeling. However, for more robustr thtreat modeling it is recommended to use some of the threat modeling tools.

There are open-source as well as commercial threat modeling tools on the market.

Open Source

- OWASP Threat Dragon
- OWASP PyTM

Commercial

- Microsoft Threat Modeling Tool (no cost)
- IriusRisk (licensed)
- ThreatModeler (licensed)

Use Case Example: Threat Model

Since we haven't developed the deployment architecture of the system yet, we still cannot yet identify all trust boundaries. With the knowledge that we already have, we can identify the Internet, the public cloud environment, the OpenShift platform, and the databases as obvious trust boundaries as they represent system boundaries.

Our information assets are:

- A01 – Contact Details
- A02 – Vehicle Details
- A03 – Payment Detail
- A04 – Identification and Authentication Information
- A05 – Payment Transaction

The Threat Actors:

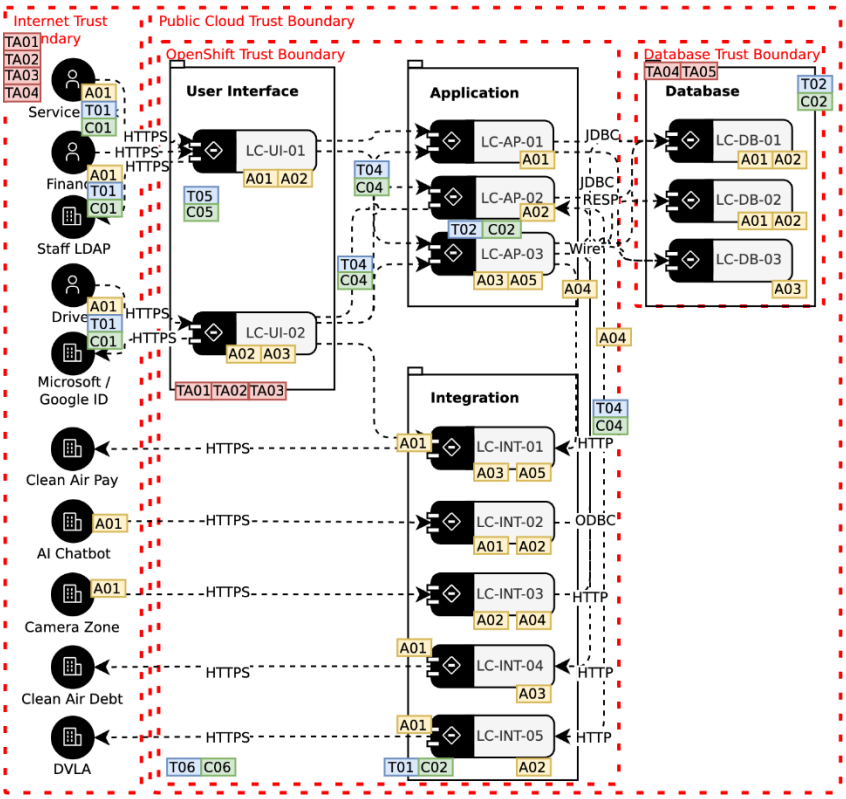
- TA01 – External Authorized
- TA02 – External unauthorized
- TA03 – Internal authorized
- TA04 – Internal unauthorized

We use the STRIDE model to identify threats and mitigating controls. The list in the following table is not comprehensive and should just illustrate the concept:

STRIDE Category	**Threat**	**Control**
Spoofing	T01 – Attackers utilize local credentials to access the Service by performing CSRF or other attacks on the users browsers.	C01 – Prevent local credential setup (preventive).
Tampering	T02 – Malicious user has direct access to a database through a management interface.	C02 – Implement separation of duties and the least privilege principle when assigning

		administrative privileges (preventive).
Repudiation	T03 – Not able to verify who created a transaction during payment.	C03 – Ensure all transactions are authenticated (preventive).
Information Disclosure	T04 – An attacker gains access to sensitive data through a man-in-the middle attack T05 - An attacker exploits a vulnerability performing a mass data exfiltration of the database	C04 – Utilize modern encryption protocols (Consider TLS1.3 and PFS ciphers) (preventive). C05.1 - Detect mass exfiltration of database (detective). C05.2 - Implement separation of duties and least privilege principle for application components authorized to query the database (preventive).
Denial of Service	T06 – Attackers perform a denial of service towards the service, preventing legitimate users from consuming the service	C06 – Configure appropriate public cloud-provided DDoS protection (protective).
Elevation of Privileges	T07 – An attacker exploits features that should be reserved for privileged users.	C07 – Implement the least privilege design principle and design privileges for users (protective).

We can now represent the specified boundaries, assets, threats, threat actors, and controls in the threat model diagram:



The risk assessment for each of the threats and their recommended protection controls should be done by our Risk Assessment Framework.

THREAT MODEL QA CHECKLIST

- Besides the functional assets in a system, we also need to identify meta data, logs, etc., which can also contain data fragments of our functional assets.
- There are various methods available to identify threats, including OWASP Top 10, STRIDE, and others. Start with a high-level view and iterate through the different levels of detail over time, depending on the location of the critical assets.
- Make sure that you have defined controls for all the threats you identified.
- Identified controls should reduce the likelihood and/or impact of the associated risk in the intended direction. If a control has no impact on either likelihood or impact, it is basically useless.

Infrastructure Security

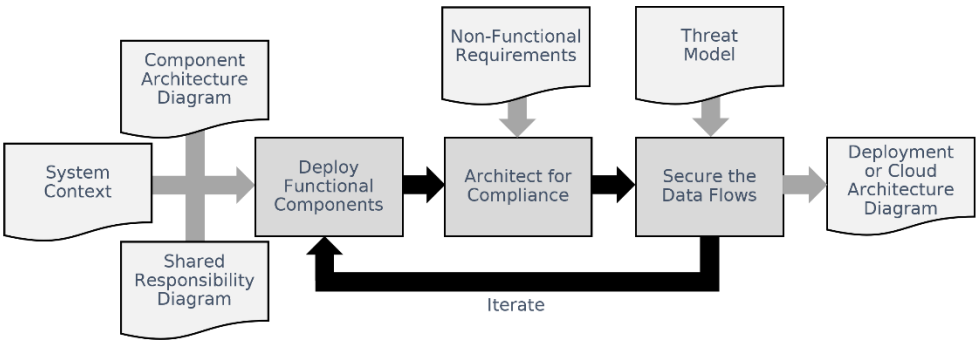
Requirements traceability is an important aspect of a solution. In the infrastructure architecture, the focus is on the non-functional requirements, including the security requirements related to

- compliance,
- general cyber risk mitigation, and
- the mitigation actions resulting from the threat modeling.

The format of the non-functional requirements traceability table could look like this:

Req. ID	Requirement	High Level Design	Low Level Design	Test Plan	Documentation

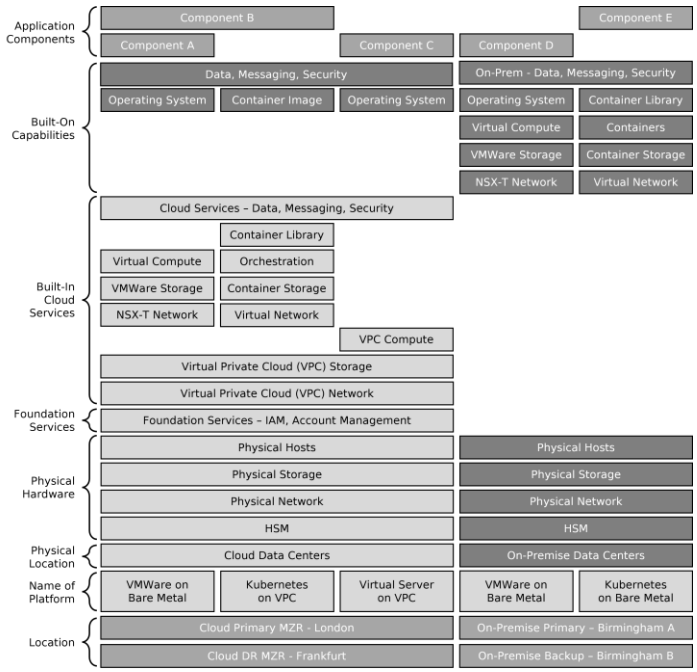
The Deployment Architecture Diagram is a foundation for designing security for infrastructure. The overall architectural thinking process flow to include security into the deployment architecture looks like this:



The three main steps in an iterative loop are:

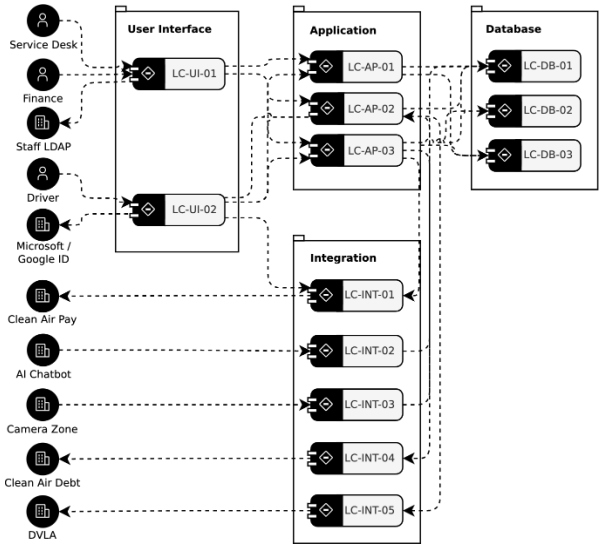
1. **Deploy Functional Components**
   First, we identify the placement of the functional components on the technology layers included in the deployment diagram. Besides application components, it may include:
   - Built-on Capabilities (e.g., operating systems, container images, containers, etc.)
   - Built-in Cloud Services (e.g., databases, storage, messaging, virtual networks, virtual private cloud compute, etc.)
   - Foundation Services (e.g., IAM, Account Manager)
   - Physical Hardware
   - Physical Location (on-premise data center, cloud data center)
   - Name of Platform (e.g., VMWare on Bare Metal, Kubernetes on VPC, etc.)
   - Location (e.g., cloud primary, cloud DR, on-remise primary, on-premise DR, etc.)

   These layers belong to the *Shared Responsibilities Stack*:



The System Context identify the interfaces to add to the deployment architecture.

First, for example, we use a component architecture diagram like this one to identify the logical technical components (nodes) without specifying a specific product or technology yet:



The next step is to place the nodes in the right location. to place them in the right location, both geographical and network location. Location is both a geographical location and a network location. Regulatory rule, latency, data sovereignty, and resiliency requirements will help us

decide on a geographic location. The network location latency and network segmentation requirements will guide us in the network design.

The next questions are about establishing network connectivity between the locations and ensuring communication security. This is about the network segmentation.

In case of a single cloud platform, we can use a cloud architecture diagram for the specific cloud platform (e.g., AWS, Azure).

2. **Architect for Compliance**
   In this step, we refine deployment architecture by bringing in the non-functional requirements including security. The requirements are usually mapped to standard capabilities. These security capabilities are often fulfilled by Integration with existing operational capabilities and the reuse of existing security services. You may not need to build new services and this might include security-related operations like threat monitoring, vulnerability management, access management, and others. So here you identify what existing services can support the requirements and adjust the deployment architecture to include points of integration.

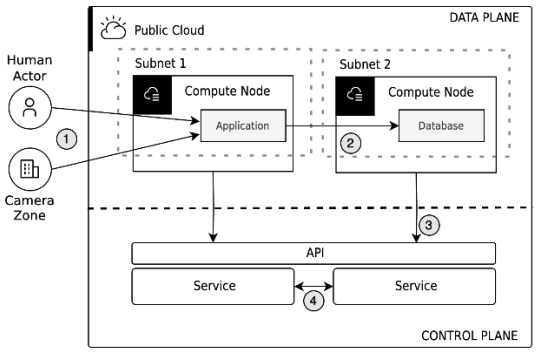
   There may be a broad set of security requirements that impact the majority of the solution components. For example, secure configuration of the infrastructure components, data encryption, etc..

   After addressing the security requirements for compliance, the next step is to look at securing the data.

3. **Secure the Data Flows**
   Data security has two parts:
   - the workload or application (that we already addressed in the application security), and
   - the underlying infrastructure

   There is a high probability that we have not completed the thread modeling as per the previous section since at that time we did not have infrastructure architecture details. In the Infrastructure Architecture context, we look at the threat modeling from four perspectives:
   - Human actor to system
   - System to system
   - System to Service and
   - Service to Service

   These four types in the following diagram:



The marked flow ① is where an external human or system actor initiates a transaction or data flow with the system. The transaction continues, marked as ②, with an interaction between two systems. The system then uses a cloud service, marked as ③ and that may cause an interaction between services, marked as ④.

Now we get into details of each of these types in order to figure out what artifacts to use:

- **Human or system actor to system**
  Transactions are initiate by either external systems or human actors. The initiated transaction transport data to a system with a compute node. The System Context diagram documents the use cases that initiates these transaction flows through
  - a user interface, or
  - synchronous API call, or
  - messaging

  You should use a session-based encryption technology, such as TLS, with mutual authentication for communication between functional components.
- **System to system**
  Processing starts on a compute node and that initiates a transaction (and data) flow to another compute node. For example, it could be an application calling a database running as a service in the cloud. In order to secure the flow, the transaction needs mutual authentication and session-level encryption.

  At either end of the flow, data is stored in file or block storage. Cloud prtoviders usually provide device-level encryption using an encryption key specific to your storage. This prevents disclosure of the data if threat actors steal the physical storage devices or when the reallocation of storage to another cloud user takes place. However, it doesn't protect data from privileged users who have access to the system and could disclose the data. You have to decide if additional levels of encryption is needed to protect against the threat of disclosure by privileged users.

  Some storage, database, or messaging products solutions, provide internal encryption at rest to protect data from privileged users, but the encryption key has to be properly protected. Writing of an unencrypted or obscured key to disk may be open to brute force attacks. The key protection mechanism is a better solution especially whe itr is achieved by a Hardware Security Module (HSM). In some cases a Software Security Module (SSM) may be an acceptable solution.

Other possible use cases for the initiation of the data flows through the system may include

  - components triggering events through errors or timed events.
  - a batch event triggered at a specific time or date.

These and all other use cases relevant to the system being built, should be added to the list of transaction (and data) flows to review.

- **System to service**
  Each compute node will use IaaS and/or platform services, such as storage and key management. They may also use PaaS services such as databases and messaging and application services (microservices). Communication normally takes place using protected APIs with identity and access management controls over mutually authenticated encrypted TLS sessions. The APIs may also use private endpoints restricted to communication within a local network.

  We also need to consider that services communicate with each other.
- **Service to Service**
  Cloud services and application services need to communicate between themselves. Within a service, there is a need to share data for resilience within an MZR and across regions. To maintain the security of the data, the same encryption in transit principles we've discussed for the other communication paths apply. You'll need to assess the security of the data communication mechanism to ensure effective security controls are in place.

Every time you make update to the architecture, you will need to perform the three-step process including

- Deploy Functional Components
- Architect the Compliance
- Secure the Data Flows

The changes could belong to deployment of new functional components, additional compliance requirements and new threats identified.

DELOYMENT VIEWPOINT QA CHECKLIST

- Ensure all human and system actors presented in the System Context diagram have communication in place on the deployment architecture diagram.
- All components in the Component Architecture diagram should be mapped to the Deployment Architecture diagram.
- Make sure that you review all architecturally significant data flows through the Deployment Architecture diagram with security controls in place.

Deployment Architecture Diagram – Case Study (Example)

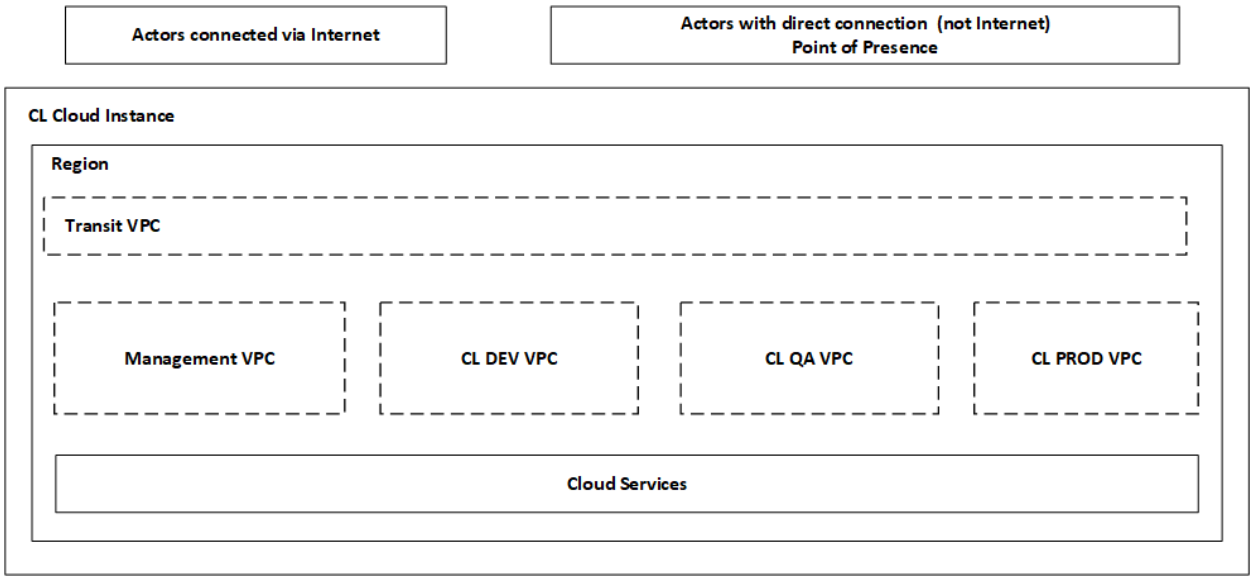
Table 8-2. Deploying functional building blocks

Type	Purpose	Technology	Component	Deployed Node
Databases	Main database	PostgresSQL	LC-DB-01	PaaS
	Cache, state data	MongoDB	LC-DB-02	PaaS
	Cache, state data	Redis DB	LC-DB-03	PaaS
Integrations	CAG → CleanAirPay	Kafka Streams	LC-INT-01	PaaS
	CAG → CleanAirDebt	RabbitMQ	LC-INT-04	PaaS
	Camera Zone → CAG	Not defined	LC-INT-03	REST API
	CAG → DVLA	Not defined	LC-INT-05	REST API
	CAG → Google / Microsoft	OAuth 2.0, OpenID	LC-UI-02	REST API
	AI Chatbot → CAG	Not defined	LC-INT-02	REST API
	CAG → Staff LDAP Directory	SAML 2.0, OpenID Connect, OAuth 2.0, and WS-Federation	LC-UI-01	PaaS
User Interfaces	Driver and Car registration	Not defined	LC-UI-02	web portal
	Payment portal	Not defined	LC-UI-02	web portal
Application Components	Driver Management	Red Hat OpenShift	LC-AP-01	Red Hat OpenShift
	Vehicle Management	Red Hat OpenShift	LC-AP-02	Red Hat OpenShift
	Payment Management	Red Hat OpenShift	LC-AP-03	Red Hat OpenShift

DEPLOYING ACROSS MANY COMPUTE PLATFORMS

In this example, we use a single compute platform but there are projects where deployment takes place across several compute platforms. This is when the architectural thinking process of deploying components onto compute platforms using the shared responsibility diagram becomes important. It ensures everyone in a team is clear on what the compute solution is for the workload.

The next iteration is the Deployment Architecture Diagram with network segments:



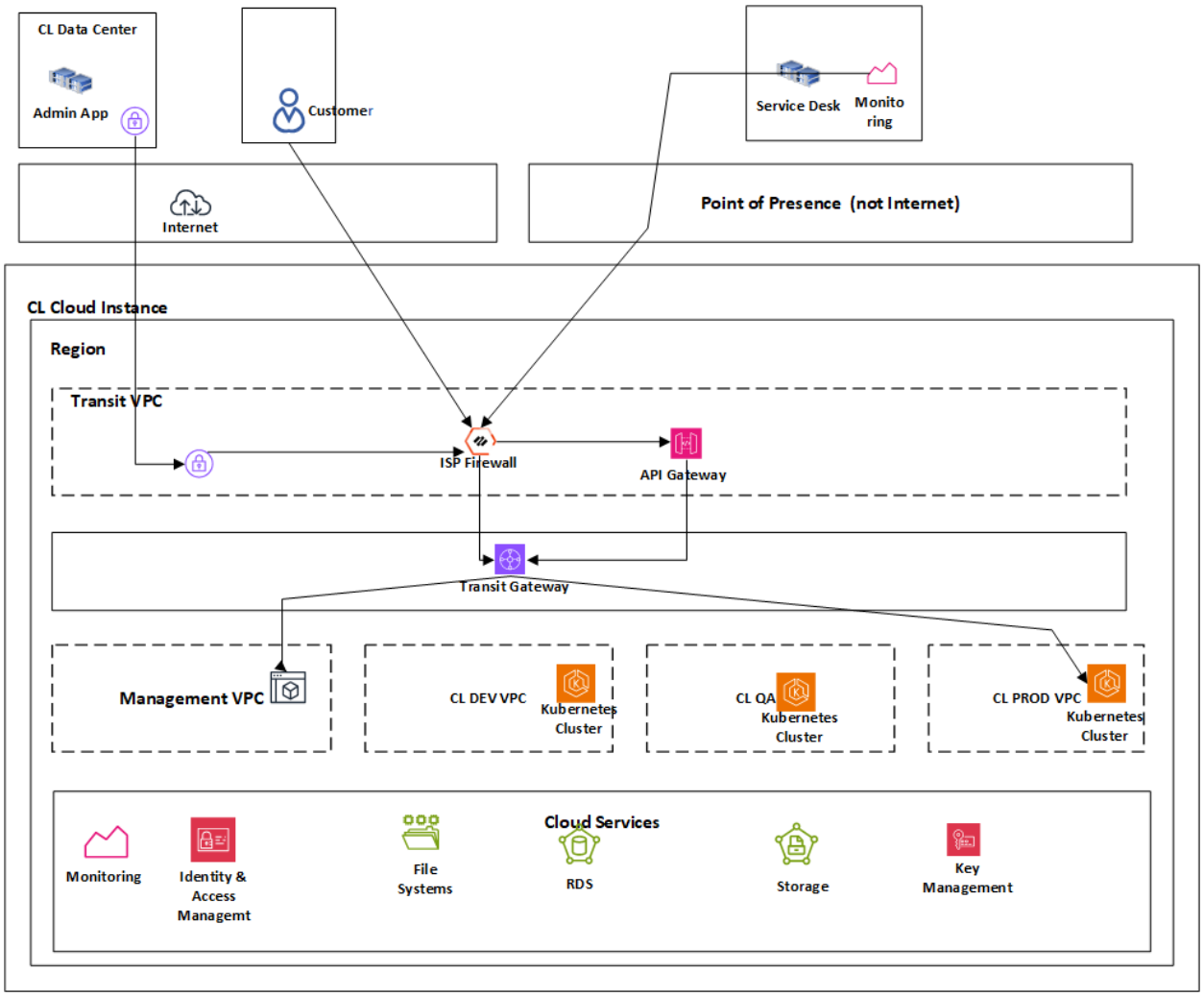
At the top, we show two ways in and out of the cloud platform. The first is the Internet, wiht connectivity to human and system actors accessible via the Internet. The second is the Point of Presence (PoP), which is the networking provider hosting private network connectivity to human and system actors.

The first layer is the company's cloud instance (the Clean Air Guildford Cloud Instance). At the top, we have the Transit Virtual Private Cloud (VPC). This is a virtual network that's private to an organization and isn't open to any network traffic by default. All traffic in and out of the cloud and between other VPCs flows through the Transit VPC.

We have a Management VPC that contains any compute hosts that manage resources on the other VPCs. Then there are three VPCs for the workload: CAG Dev VPC for application development, CAG Test VPC for testing, and CAG Prod VPC for running the workload in production.

At the bottom, we have the collection of cloud services provided by the cloud provider.

Now, we add the functional components to the Deployment Architecture diagram:



Now, we will explain the parts of the Deployment Architecture diagram:

- **Transit VPC**
  In the Transit VPC, we have the NextGen Firewall that controls all traffic in and out of the cloud platform flows. On the left, we've got a VPN termination point that could be a cloud service or a part of the NextGen Firewall. Internet connectivity using VPNs terminates at this point. On the right, we've got an API Gateway that protects API access for the application. Just below the Transit VPC is a Transit Gateway that joins the VPCs networks.
- **Management VPC**
  In the Management VPC, we've got a Bastion Host that provides support for remote administration. Administrators must first login to this server before accessing any other resource in the cloud. The server automatically retrieves usernames and passwords from the Privileged Access Management server for login to the hosted cloud resources. In this case, it's a software solution and not a cloud service. In the event of an investigation, the bastion host will record the activity of a remote administrator.
- **CAG VPCs**

We've got three workload VPCs for the Clean Air Guildford (CAG) application, with one for development, one for testing, and one for production. In each VPC, there is an instance of Red Hat OpenShift for running the application.

- **Platform services**
  On the left-hand side of the platform services, we've used the standard Logging, Monitoring and DNS Services provided by the cloud provider. The solution required an LDAP directory for employees and so we've also used that as a cloud service rather than running our own software. There are other services illustrated on the diagram as well.

  The VPC network will have private endpoints for each of the cloud services, ensuring there is no public network connectivity. This removes the possibility of exposing the cloud services to the Internet.
- **Internet Connectivity**
  At the top left is the internet connectivity. The case study said the Debt Collection App and the Payment Gateway are available via the Internet using a VPN connection to secure access. A third Internet connection enables the drivers to access the portal for the administration of the drivers, vehicles, and payments.
- **Point of Presence Connectivity**
  At the top right is private connectivity through the Point of Presence (PoP). Third-party networking service providers typically manage the PoP, which enables connectivity to other networks. The data doesn't flow over the internet and in this case, we didn't use a site-to-site VPN to protect the confidentiality of the data.

We may identify additional cloud services needed to support the application operations, and the Deployment Architecture diagram will evolve as we go through design iterations.

Now we'll move to a further discussion on zero trust practices for the next iteration of the Deployment Architecture diagram.

Zero Trust Based Security Infrastructure

There are two most common zero trust networking solutions:

- **Zero Trust Network Access (ZTNA)**
  ZTNA is mostly used to control network traffic between an end-user and a an enterprise application. An enterprise application is an application that's only accessible to employees and possibly some partners of an organization. ZTNA is a brokered network connection that uses an encrypted micro-tunnel to connect a device to the system hosting the application. It only permits network connectivity to the port that has been explicitly defined by policies and blocks all other traffic.

  ZTNA is the most common implementation of the Software Defined Perimeter (SDP) pattern and allows organizations to define logical perimeters to protect private applications. While ZTNA could use both human and system actors, in practice organizations select ZTNA primarily to manage end-user access. An important advantage of ZTNA is that it isolates the applications from the internet, thus reducing the attack surface of the organizations. The use cases where ZTNA comes into the picture are:
    - replacement of remote access VPN,
    - acceleration of cross-organization application access during a merger or acquisition, and

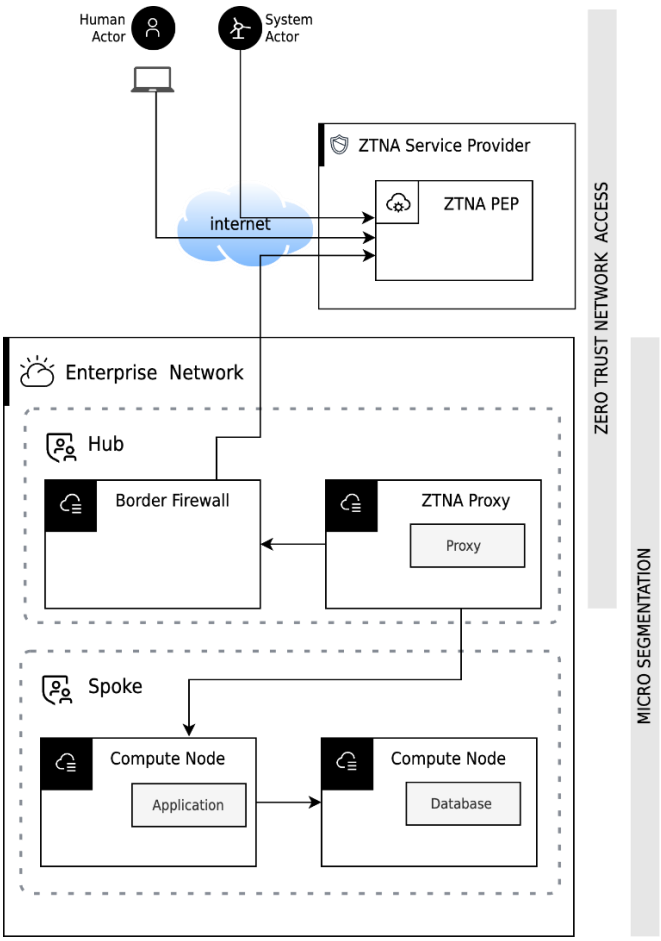
- multi-cloud access.

With ZTNA, you can define a policy for every type of network connectivity between the end-user device and the internal application. As a result, possible malware on the end-user device can no longer propagate to other systems using protocols like SSH, WinRM, or RDP.

Another advantage of ZTNA is that most vendors deliver the ZTNA solution as part of Secure Access Service Edge (SASE), which provides extra security controls at the Policy Enforcement Point like traffic inspections, for example. This multi-faceted security check at the PEP reduces traffic redirection to different types of security components.

- **Microsegmentation**
  Micro-segmentation limits the communication between applications and related systems within one or more network zones. It restricts the network to the TCP/IP ports needed by the applications through explicitly defined policies, and it inhibits all other traffic. There are several approaches possible to micro-segmentation, mostly by packet filters at the endpoint level or at the network level.

Both solutions allow network communication only if an explicit policy permits such connectivity. ZTNA establishes a software-defined perimeter around a set of applications and manages access by end-users. Micro-segmentation focuses on system-to-system network connectivity. ZTNA mostly provides a logical perimeter for front-end applications and micro-segmentation controls the communication between computing nodes themselves.



Endpoint based solutions

Applying the “assume breach” principle to an endpoint is about the implementation of as many layers of defense as possible on the endpoint itself. It starts with protective controls like host-based firewalls, host-based intrusion prevention solutions, and access control. Additionally, there should be detective controls, ideally combined with a local response capability.

Endpoint Detection and Respond (EDR) solutions are important parts. Even with all possible protective controls in place, the assume breach principle implies not only the compromise of surrounding systems but also the endpoint itself. It means that a detection capability is key. First to the initiation of the necessary response actions to restore the endpoint to its original state, but also to detect issues with other solutions as contextual information. The EDR solution may deny other resources to the end-user or application running on a compromised endpoint.

Also, there are even more dynamic measurements possible of the security posture based on EDR solutions. These security products calculate a risk score based on both static and dynamic data. For example, if there is detection at the endpoint of some suspicious behavior, the risk score will increase. Other zero trust based solutions, like ZTNA, take this risk score as extra contextual information for the evaluation of access policies. Denial of access to the network can take place if devices have a risk score that’s too high.

Identity and Access Management

Identity and access management (IAM) are capabilities we need for each IT layer in the organization. So beside the infrastructure layer, IAM plays a key role for application and cloud security too.

IAM has become a security foundational element.

In the context of zero trust, it's clear that identity is a core element of many zero trust based solutions. All actors requiring access will require a unique identity and many resources will have an identity too. The verification of the identity is the recurring baseline for zero trust. An employee will only get access to an internal application after validation of their identity, including all contextual information. That might include historical data on both employee login times and locations.

Privileged Access Management (PAM) provides strict control over administrator-equivalent access to systems and applications. Applying zero trust to such types of solutions is mostly based on additional conditions before granting access. For example, before a system administrator can access a system via PAM, there should be a service ticket in the service management tool. Additionally, the granted access could have a limited duration. This is how least privilege reduces further the implicit permissions.

Lastly, the recent development of Identity Threat Detection and Response (ITDR) solutions is another layer in the IAM portfolio. In a hybrid cloud context with many more identities to manage and the rise of threats to identity tools themselves, there is a need for a detection capability similar to what EDR does for the endpoint. ITDR will be another key zero trust solution to maintain trustworthy identities and detect possible anomalies with them.

Architecting Zero Trust Practices

Now we should go back to the sequence diagrams and data flow diagrams and updated threat model and security controls walking through these diagrams in the context of the Deployment Architecture diagram.

1. Return to the threat modeling documented in the Application Security section and perform an update of the STRIDE threats and controls identified in the STRIDE Threats and Controls table.
2. Identify any zero-trust practices for adding to the STRIDE threats and controls table. Update and add any relevant risk evaluations.
3. Take each of the identified controls and update the requirements, taking care to define requirements that can help identify the technical and process solutions needed to implement the controls.
4. Take the requirements and identify their implementation by updating requirements mappings that document their implementation.
5. Update architecture diagrams to reflect changes, starting with the system context diagram with any new actors. Update the deployment architecture diagrams as needed. If you have selected a ZTNA solution, there may be a need to create a standalone solution architecture to provide sufficient detail. It's likely to require consideration of non-functional requirements such as availability and performance.

6. Remove implicit trusts where possible. For example, system A has network connectivity to system B because both systems reside on the same VLAN or network zone (for example, the production environment), while these two systems don't require connectivity between them.
7. You might discover new security services that require management by a security operations team, like ZTNA.

Zero-trust-based solutions won't protect us against all the possible threats that we may be facing. It is important that you understand the limitations of zero-trust-based solutions. A good example is a software vulnerability in an internet-facing web application. That isn't something you will solve with adaptive access control or micro-segmentation, just to name two common zero-trust-based solutions.

ZERO TRUST BASED SECURITY INFRASTRUCTURE QA CHECKLIST

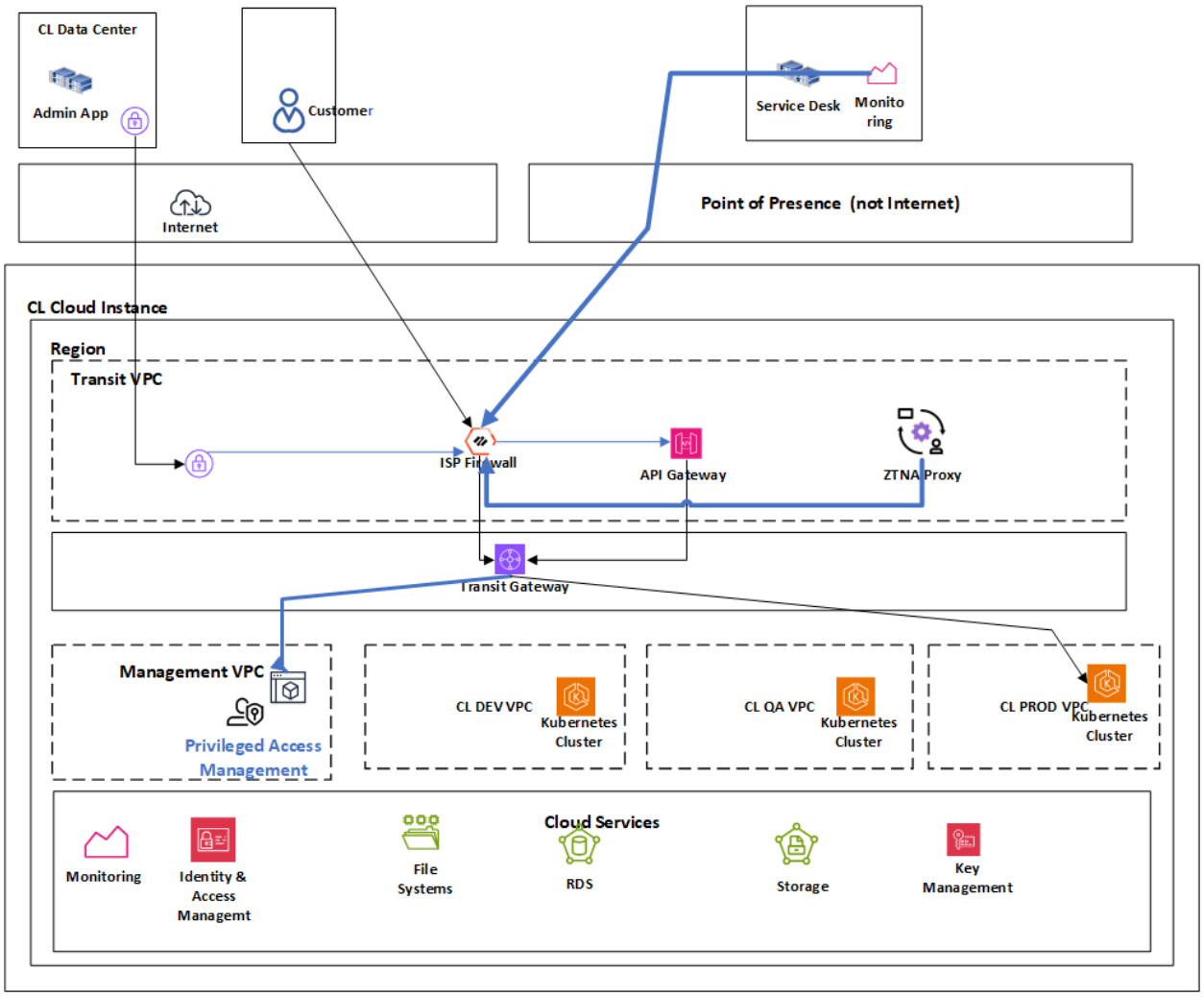
- Ensure solution includes network-based solutions using ZTNA and micro-segmentation.
- Review and ensure the inclusion of endpoint based solutions.
- Review and ensure the inclusion of identity and access management solutions.
- Use the list of zero trust practices to review the deployment architecture diagram.

Case Study (Zero Trust)

This example is about implementing ZTNA ad PAM:

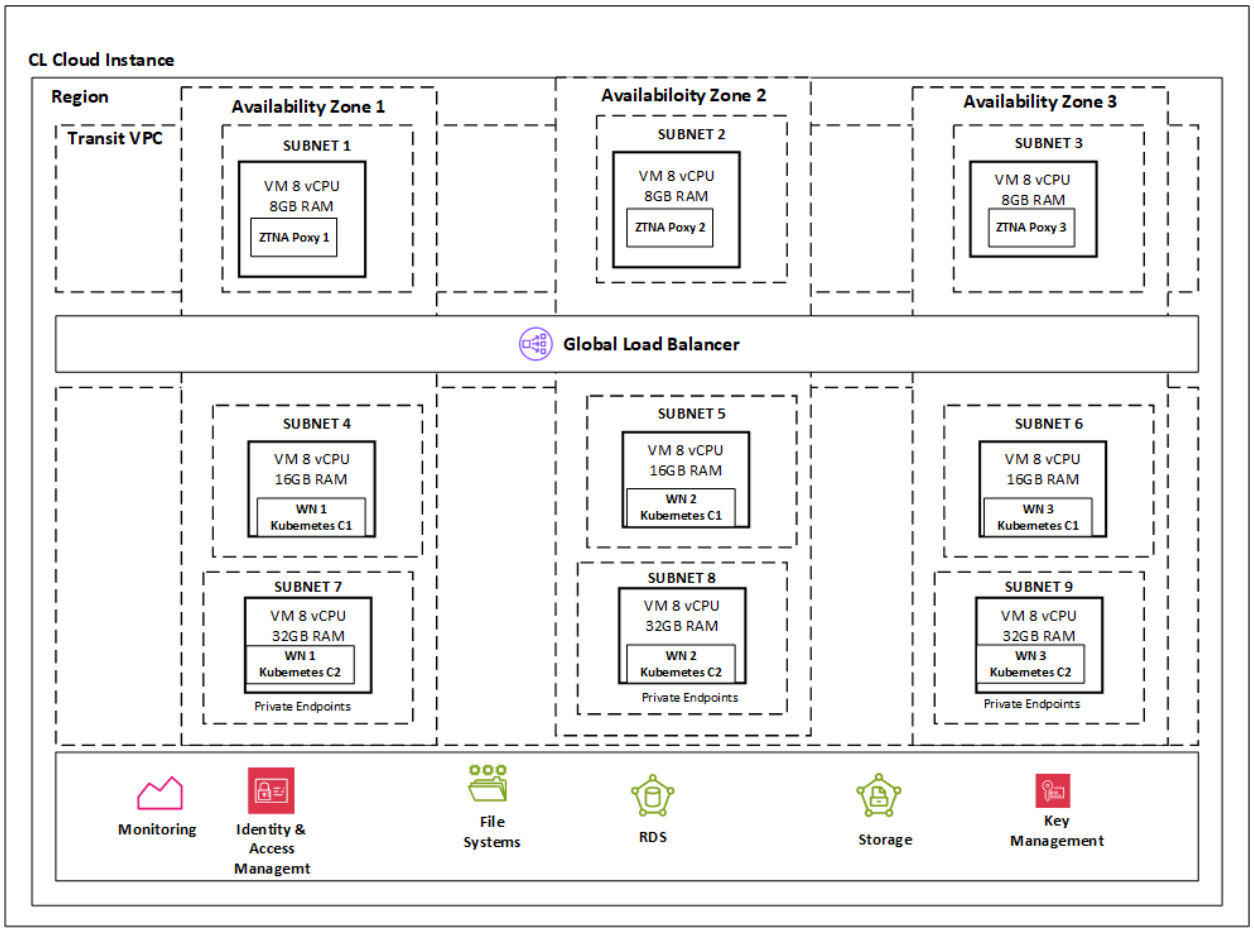
- **Implement a ZTNA solution**
  There will be a ZTNA remote access SaaS service added for the Clear Air Guildford workforce and business partners and the Transit VPC will include a proxy to support access to the cloud platform. A decision was made to use a SaaS service as they can provide a more flexible and available service than we could implement with software.
- **Add Privileged Access Management (PAM)**
  A Privileged Access Management (PAM) solution is added to control all privileged access to the CAG systems and applications. We've decided to run as software on the Management VPC as the software we require isn't available from the cloud provider.

Implementing a PAM solution is part of an iterative approach to architecting security, wherein you gradually add more capabilities to the solution. A possible step you could take to reach even higher maturity could be through the integration of the PAM solution with the service desk solution. Starting from that moment, an administrator will only be able to check out a privileged account from the PAM system if there is an approved ticket in the service management solution. This just-in-time approach is a more granular access control for privileged access as it establishes a dependency between service management and administrative access to systems.



Cloud Architecture Diagram

A full cloud architecture diagram is usually a complex diagram and we selected just one aspect of the solution: Zero Trust Network Access to the admin console of the application for the staff. For example, the Helpdesk service. This web-based admin application isn't accessible to the public, though it should be accessible from the Internet in a secure way.



The following components from the previous diagram support the application:

- **ZTNA proxy**
  ZNTA proxy, hosted on a virtual server, is deployed in a redundant way over the three availability zones in the Transit VPC. The ZTNA proxy will establish sessions to the Admin web application via a load balancer.
- **Load balancer**
  The load balancer manages the traffic between the ZNTA proxies and the Admin Web Application running in the CL Prod VPC in subnets 4, 5 and 6.
- **Admin web application**
  This administrator console is running on a Kubernetes cluster in the CL Prod VPC and deployed in a redundant way over the three availability zones in subnets 4, 5 and 6.
- **The back-end application**
  The back-end application is running on another Kubernetes cluster in a dedicated subnet in the CP Prod VPC in subnets 7, 8 and 9. In the same subnet as the back-end app are the private endpoints for a direct network connection to the databases.

We should draw multiple viewpoints to show a perspective of the application.

CLOUD ARCHITECTURE DIAGRAM QA CHECKLIST

- For the support of zero trust, review cloud services for support of encryption in transit, mutual authentication and automated rotation of certificates.
- Match the availability characteristics of the security solutions to the those of the workload or application you are securing.
- Ensure the availability characteristics of the cloud services supporting security solutions.
- Check data flows for both primary and failover routes for protection.
- Check all compute nodes have appropriate ZTNA agents and controls in place.

Suggested reading:
Zero Trust Architecture (nist.gov)
A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Location Environments (nist.gov)
Secure-by-Design | CISA
Cybersecurity Framework | NIST
Enterprise Security Architecture (oreilly.com)
3 Enterprise Security Architecture (opengroup.org)
Cloud Controls Matrix and CAIQ v4 | CSA (cloudsecurityalliance.org)
Threat Modeling: Designing for Security (oreilly.com)
Threat Modeling (oreilly.com)
MITRE ATT&CK framework
Elevation of Privilege (EoP) card game for threat modeling
OWASP Top 10
LINDDUN Privacy Threat Modelling Framework
OWASP Proactive Controls | OWASP Foundation
Envoy proxy - home

Risk methods:
The Importance and Effectiveness of Cyber Risk Quantification (fairinstitute.org)
The ISF is a leading authority on information security and risk management - Information Security Forum
OWASP Risk Rating Methodology | OWASP Foundation
OWASP RISK RATING CALCULATOR (owasp-risk-rating.com)